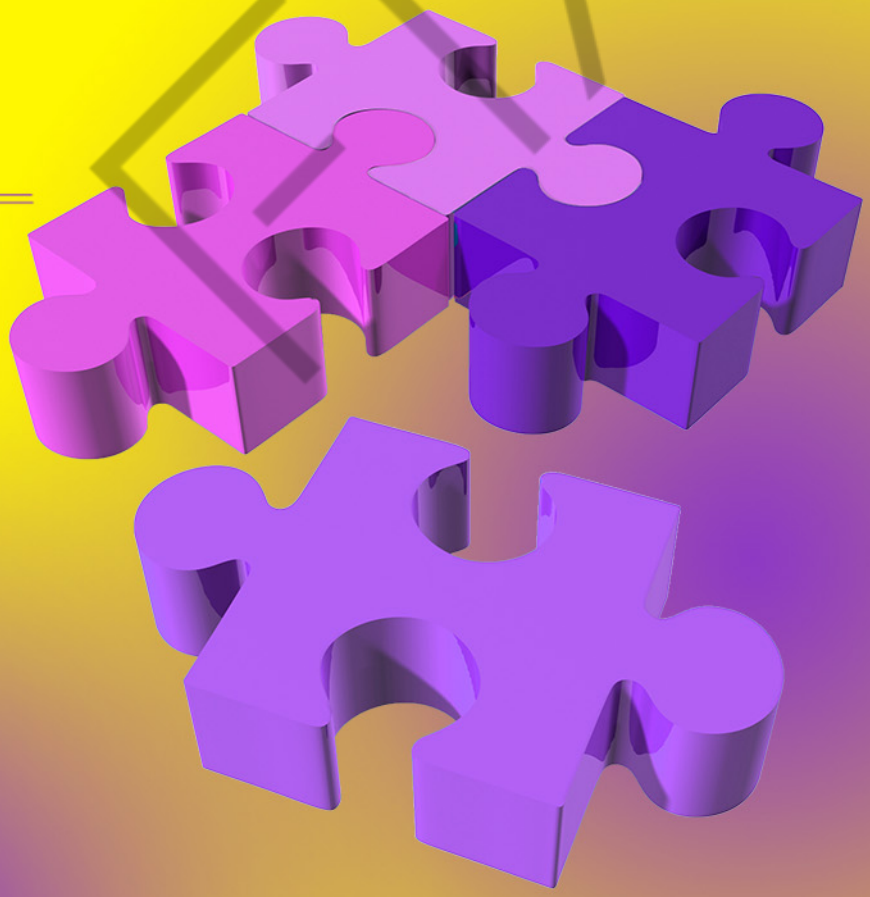
START DATA MANAGEMENT

SQL: COMANDOS DQL/DRS - JUNÇÃO DE TABELAS



11

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE FIGURAS

Figura 1 – Modelo E-R - Relacionamento 18
Figura 2 – Ilustração de junção de tabelas 19
Figura 3 – Resultado da consulta às tabelas DEPARTAMENTO e FUNCIONARIO sem ALIAS 21
Figura 4 – Resultado da consulta às tabelas DEPARTAMENTO e FUNCIONARIO com ALIAS 22
Figura 5 – Cross Join 23
Figura 6 – Conteúdo da tabela DEPARTAMENTO 23
Figura 7 – Conteúdo da tabela FUNCIONARIO 23
Figura 8 – Resultado do CROSS JOIN 24
Figura 9 – INNER JOIN 25
Figura 10 – Resultado do INNER JOIN 27
Figura 11 – Resultado do INNER JOIN com USING 28
Figura 12 – Resultado do INNER JOIN com várias tabelas 30
Figura 13 – LEFT OUTER JOIN 32
Figura 14 – LEFT OUTER JOIN com WHERE NULL 32
Figura 15 – Resultado do LEFT OUTER JOIN 33
Figura 16 – Resultado do LEFT OUTER JOIN, recuperando NULOS 34
Figura 17 – RIGHT OUTER JOIN 35
Figura 18 – RIGHT OUTER JOIN com WHERE NULL 36
Figura 19 – Resultado do RIGHT OUTER JOIN 37
Figura 20 – Resultado do RIGHT OUTER JOIN, recuperando NULOS 38
Figura 21 – FULL OUTER JOIN 39
Figura 22 – FULL OUTER JOIN com WHERE NULL 39
Figura 23 – Modelo de dados com relacionamento opcional 40
Figura 24 – Conteúdo armazenado na tabela EMPRESA CONVENIADA 45
Figura 25 – Conteúdo armazenado na tabela MATRICULA 45
Figura 26 – Resultado do FULL OUTER JOIN 46
Figura 27 – Resultado do FULL OUTER JOIN, recuperando NULOS 46
Figura 28 – Modelo de dados, autorrelacionamento 48
Figura 29 – Conteúdo armazenado na tabela FUNCIONARIO 51
Figura 30 – Resultado do SELF JOIN 52
Figura 31 – Modelo de dados, sem relacionamento entre as tabelas 53
Figura 32 – Conteúdo armazenado na tabela CD 56
Figura 33 – Conteúdo armazenado na tabela FAIXA_PRECO 56
Figura 34 – Resultado do NON-EQUIJOIN 57
Figura 35 – Resultado da Consulta 1 58
Figura 36 – Resultado da Consulta 2 59
Figura 37 – Resultado da Consulta 3 60
Figura 38 – Resultado da Consulta 4 61

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de código-fonte SQL – comando SELECT utilizando junções 8
Código-fonte 2 – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS 11
Código-fonte 3 – Script para popular as tabelas do banco de dados SIP..... 17
Código-fonte 4 – Exemplo de código-fonte SQL–comando SELECT com qualificador de nomes..... 19
Código-fonte 5 – Exemplo de código-fonte SQL-Comando SELECT com qualificador de nomes..... 20
Código-fonte 6 – Exemplo de código-fonte SQL – comando SELECT sem ALIAS de tabela..... 20
Código-fonte 7 – Exemplo de código-fonte SQL – comando SELECT com ALIAS de tabela..... 21
Código-fonte 8 – Exemplo de código-fonte SQL – CROSS JOIN..... 24
Código-fonte 9 – Exemplo de código-fonte SQL – INNER JOIN 26
Código-fonte 10 – Exemplo de código-fonte SQL – INNER JOIN com USING 28
Código-fonte 11 – Exemplo de código-fonte SQL – INNER JOIN – Várias tabelas.. 29
Código-fonte 12 – Exemplo de código-fonte SQL – INNER JOIN com USING 30
Código-fonte 13 – Exemplo de código-fonte SQL – LEFT OUTER JOIN 33
Código-fonte 14 – Exemplo de código-fonte SQL – LEFT OUTER JOIN, recuperando NULOS 34
Código-fonte 15 – Exemplo de código-fonte SQL – LEFT OUTER JOIN com USING 34
Código-fonte 16 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN..... 36
Código-fonte 17 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN, recuperando NULOS..... 37
Código-fonte 18 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN com USING 38
Código-fonte 19 – Exemplo de código-fonte SQL – SCRIPT para criar estrutura exemplo FULL OUTER JOIN 44
Código-fonte 20 – Exemplo de código-fonte SQL – FULL OUTER JOIN 45
Código-fonte 21 – Exemplo de código-fonte SQL – FULL OUTER JOIN, recuperando NULOS 46
Código-fonte 23 – Exemplo de código-fonte SQL – FULL OUTER JOIN com USING 47
Código-fonte 23 – Exemplo de código-fonte SQL – Script para criar a estrutura para o SELF JOIN 51
Código-fonte 24 – Exemplo de código-fonte SQL – SELF JOIN..... 51
Código-fonte 25 – Exemplo de código-fonte SQL – Script para criar estrutura para NON-EQUIJOIN 56
Código-fonte 26 – Exemplo de código-fonte SQL – NON-EQUIJOIN..... 56
Código-fonte 27 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 1 58
Código-fonte 28 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 2 59
Código-fonte 29 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 3 60

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Código-fonte 30 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 4 61

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

SUMÁRIO

1 COMANDOS DRS – JOIN 6
1.1 Junção de Tabelas 6
1.2 Condições de Join 7
1.2.1 Tipos de Join 7
1.2.2 Comando SELECT utilizando junções 7
1.3 Exemplos de aplicação do comando SELECT utilizando junções 8
1.4 Pesquisa em múltiplas tabelas 18
1.4.1 Qualificadores de Nome 19
1.4.2 ALIAS – Apelido para tabelas 20
1.4.3 Cross Join – Junção Cruzada 22
1.4.4 Inner Join – Junção Interna 25
1.4.5 Inner Join com cláusula Using – Junção Interna 27
1.4.6 Inner Join com duas ou mais tabelas – Junção Interna 28
1.5 Junções Externas 31
1.5.1 Left Outer Join – Junções Externas 31
1.5.2 Right Outer Join – Junções Externas 35
1.5.3 Full Outer Join – Junções Externas 38
1.6 Self Join ou Autojunção 47
1.7 Non-equijoins – Junções não idênticas 52
1.8 Hands On – Consulta ou Recuperação de dados utilizando JOINS 57
1.8.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS 57
1.8.1.1 Recuperando dados utilizando JOINS 57
REFERÊNCIAS 62
GLOSSÁRIO 63

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1 COMANDOS DRS – JOIN

A principal função de um sistema de banco de dados é proporcionar recursos para armazenamento, manutenção e consulta dos dados, de maneira segura e eficaz.

A **consulta de dados é uma pesquisa** realizada **no banco** de dados, cujo propósito é recuperar as linhas (registros) **que atendem** às **condições pré-estabelecidas**. Iremos trabalhar as consultas que recuperam dados de duas ou mais tabelas, que chamamos de junções.

A instrução utilizada para a realização das consultas é o **SELECT**, que faz parte da categoria de operações DML (Data Manipulation Language);

Alguns autores criam um grupo a parte para tratar somente do SELECT, o **DQL**, Data Query Language (**Linguagem de Consulta de Dados**) ou **DRS**, Data Retrieve Language (**Linguagem de Recuperação de Dados**).

1.1 Junção de Tabelas

Na linguagem SQL temos instruções que nos permitem recuperar os dados de tabelas de um banco de dados. Essas instruções pertencem à divisão DRS (*Data Retrieve Statement*) ou DQL (*Data Query Language*) da linguagem SQL.

O comando SELECT a partir de junções (JOIN) nos permite a pesquisa em múltiplas tabelas. Esta pesquisa é necessária quando precisamos recuperar dados que estão distribuídos em várias tabelas.

A operação JUNÇÃO (JOIN) é usada para combinar registros e linhas relacionados dentro de um único registro/linha. Esta operação nos permite processar os relacionamentos entre os registros (linhas) das tabelas.

O critério de relacionamento estabelecido pode ser baseado em junções idênticas (equijoins), não idênticas (non-equijoins), internas (inner join), externas (left join, right join ou full join) ou autojunções (self-join).

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.2 Condições de Join

Há dois tipos de condições de join baseadas no operador utilizado:

- **EQUIJOINS:** utilizam o operador de igualdade (=). São consideradas como consequência de um modelo relacional, ou seja, espelham a relação dos dados representados em um modelo E-R. São as relações em que temos a presença das chaves primária e estrangeira.
- **NON-EQUIJOINS:** utilizam um operador que não é o de igualdade, como <, >, BETWEEN etc. São consideradas como junções atípicas, em que não há a presença de chave primária e nem mesmo chaves estrangeiras. Os relacionamentos são feitos por meio de colunas, normalmente não chaves primárias e nem únicas, mas que possuem algum tipo de relação associada à necessidade de negócio, para que possamos expressar a junção.

1.2.1 Tipos de Join

Há três tipos diferentes de JOINS:

- **Joins Internas:** retornam um registro (linha) somente quando as colunas da junção tiverem valores que satisfaçam a condição da junção.
- **Joins Externas:** retornam um registro (linha) mesmo quando uma das colunas na condição de junção tiver um valor nulo.
- **Autojoins:** retornam registros (linhas), possuem valores associados na mesma tabela.

1.2.2 Comando SELECT utilizando junções

Para recuperarmos os dados das tabelas de um banco de dados, utilizamos o comando SELECT, que é um comando dentro da divisão DRS/DQL da linguagem SQL.

Sintaxe básica do comando SELECT utilizando junções:

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
SELECT tabela1.coluna, tabela2.coluna
FROM tabela1
    [CROSS JOIN tabela2] |
    [NATURAL JOIN tabela2] |
    [JOIN tabela2 USING (nome_coluna)] |
    [JOIN tabela2
    ON(tabela1.nome_coluna = tabela2.nome_coluna)] |
    [LEFT|RIGHT|FULL OUTER JOIN tabela2
    ON (tabela1.nome_coluna = tabela2.nome_coluna)];
```

Código-fonte 1 – Exemplo de código-fonte SQL – comando SELECT utilizando junções
Fonte: Price (2009)

1.3 Exemplos de aplicação do comando SELECT utilizando junções

Para exemplificar as aplicações dos comandos DRS/DQL, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_PROJETO_SIP.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados do PROJETO SIP.

```
-- SCRIPT PARA CRIAÇÃO DO BANCO DE DADOS

-- DO ESTUDO DE CASO SIP

-- COMANDOS PARA EXCLUIR AS TABELAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
DROP TABLE t_sip_dependente CASCADE CONSTRAINTS;
DROP TABLE t_sip_funcionario CASCADE CONSTRAINTS;
DROP TABLE t_sip_implantacao CASCADE CONSTRAINTS;
DROP TABLE t_sip_projeto CASCADE CONSTRAINTS;
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
-- COMANDOS PARA CRIAR AS TABELAS

-- E ADICIONAR CONSTRAINTS: PRIMARY KEY, CHECK E UNIQUE

CREATE TABLE t_sip_departamento (
    cd_depto NUMBER(3)       NOT NULL,
    nm_depto VARCHAR2(30)    NOT NULL,
    sg_depto CHAR(3)         NOT NULL
);

COMMENT ON COLUMN t_sip_departamento.cd_depto IS
    'Um departamento é identificado por um código numérico de
três dígitos.';

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT pk_sip_departamento
   PRIMARY KEY ( cd_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_nome
        UNIQUE ( nm_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_sigla
        UNIQUE ( sg_depto );

CREATE TABLE t_sip_dependente (
    nr_matricula  NUMBER(5)      NOT NULL,
    cd_dependente NUMBER(2)      NOT NULL,
    nm_dependente VARCHAR2(50)   NOT NULL,
    dt_nascimento DATE           NOT NULL
);

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT pk_sip_dependente
   PRIMARY KEY ( nr_matricula, cd_dependente );

CREATE TABLE t_sip_funcionario (
    nr_matricula      NUMBER(5)      NOT NULL,
    cd_depto          NUMBER(3)      NOT NULL,
    nm_funcionario    VARCHAR2(50)   NOT NULL,
    dt_nascimento     DATE           NOT NULL,
    dt_admissao       DATE           NOT NULL,
    ds_endereco       VARCHAR2(100)  NOT NULL,
    vl_salario_mensal NUMBER(7, 2)  NOT NULL
);

   ALTER TABLE t_sip_funcionario
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
ADD CONSTRAINT ck_sip_func_salario
         CHECK ( vl_salario_mensal >= 1212 );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT pk_sip_funcionario
   PRIMARY KEY ( nr_matricula );

CREATE TABLE t_sip_implantacao (
    cd_projeto     NUMBER(5) NOT NULL,
    cd_implantacao NUMBER(3) NOT NULL,
    nr_matricula   NUMBER(5) NOT NULL,
    dt_entrada     DATE      NOT NULL,
    dt_saida       DATE          NULL
);

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT ck_sip_implant_data
         CHECK ( dt_saida > dt_entrada );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT pk_sip_implantacao
   PRIMARY KEY ( cd_projeto, cd_implantacao );

CREATE TABLE t_sip_projeto (
    cd_projeto NUMBER(5)    NOT NULL,
    nm_projeto VARCHAR2(40) NOT NULL,
    dt_inicio  DATE         NOT NULL,
    dt_termino DATE
);

-- ADICIONAR CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT ck_sip_projeto_data
         CHECK ( dt_termino > dt_inicio );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT pk_sip_projeto
   PRIMARY KEY ( cd_projeto );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT un_sip_projeto_nome
        UNIQUE ( nm_projeto );

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT fk_sip_dependente_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT fk_sip_func_depto
   FOREIGN KEY ( cd_depto )
    REFERENCES t_sip_departamento ( cd_depto );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_projeto
   FOREIGN KEY ( cd_projeto )
    REFERENCES t_sip_projeto ( cd_projeto );
```

Código-fonte 2 – SCRIPT DE CRIAÇÃO DO BANCO DE DADOS
Fonte: Elaborado pela autora (2023)

Iremos popular toda a estrutura do banco de dados referente ao projeto SIP.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DML_PROJETO_SIP.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite popular a estrutura de banco de dados do PROJETO SIP.

```
-- SCRIPT PARA POPULAR O BD IMPLANTACAO DE PROJETOS

-- ELIMINA O CONTEUDO DAS TABELAS
/*
DELETE FROM T_SIP_IMPLANTACAO;
DELETE FROM T_SIP_PROJETO;
DELETE FROM T_SIP_DEPENDENTE;
DELETE FROM T_SIP_FUNCIONARIO;
DELETE FROM T_SIP_DEPARTAMENTO;
*/

-- POPULAR A TABELA DEPTO COM 5 DEPARTAMENTOS
INSERT INTO T_SIP_DEPARTAMENTO
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
     VALUES (1, 'FINANCEIRO', 'FIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (2, 'TECNOLOGIA DA INFORMAÇÃO','TIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (3, 'CONTAS A PAGAR','CPG');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (4, 'FATURAMENTO','FAT');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (5, 'RECURSOS HUMANOS', 'RHU');

-- POPULAR A TABELA FUNCIONARIO COM 3 FUNCIONARIOS PARA CADA
DEPTO
-- DEPTO = 1
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12345,         1,         'JOAO         DA         SILVA',
TO_DATE('10/05/1985','DD/MM/YYYY'),
     TO_DATE('15/09/2012','DD/MM/YYYY'),'RUA X, 49', 5684.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12346,         1,         'MANUEL        DA         SILVA',
TO_DATE('05/10/1998','DD/MM/YYYY'),
     TO_DATE('10/11/2015','DD/MM/YYYY'),'RUA X, 31', 3542.11);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12347,         1,         'JANDIRA       DA         SILVA',
TO_DATE('10/12/2000','DD/MM/YYYY'),
     TO_DATE('15/09/2018','DD/MM/YYYY'),'RUA X, 25', 1875.96);

-- DEPTO = 2
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12348,         2,         'KATIA        REGINA        SOUZA',
TO_DATE('15/01/1995','DD/MM/YYYY'),
     TO_DATE('03/10/2015','DD/MM/YYYY'),'RUA Y, 49', 3894.63);
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12349,      2,      'MARIA      DAS      DORES      SOUZA',
TO_DATE('18/08/1983','DD/MM/YYYY'),
    TO_DATE('23/10/2017','DD/MM/YYYY'),'RUA Y, 35', 1542.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12350,        2,        'ALFREDO        DE        SOUZA',
TO_DATE('04/05/1999','DD/MM/YYYY'),
    TO_DATE('05/06/2014','DD/MM/YYYY'),'RUA Y, 27', 5874.52);

-- DEPTO = 3
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12351,        3,        'GISELE        DE        JESUS',
TO_DATE('15/04/1999','DD/MM/YYYY'),
    TO_DATE('20/03/2017','DD/MM/YYYY'),'RUA Z, 49', 1360.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12352,        3,        'RAFAEL        DE        JESUS',
TO_DATE('10/08/1998','DD/MM/YYYY'),
    TO_DATE('10/08/2012','DD/MM/YYYY'),'RUA Z, 55', 2563.44);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12353,        3,        'ROSANA        DE        JESUS',
TO_DATE('14/03/1987','DD/MM/YYYY'),
    TO_DATE('15/08/2019','DD/MM/YYYY'),'RUA Z, 79', 4879.55);

-- DEPTO = 4
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12354,       4,       'JOSEFINA       DE       ALMEIDA',
TO_DATE('16/10/1997','DD/MM/YYYY'),
    TO_DATE('25/03/2013','DD/MM/YYYY'),'RUA Y, 33', 4561.88);

INSERT INTO T_SIP_FUNCIONARIO VALUES
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
    (12355,        4,        'LUCIANA        DE        ALMEIDA',
TO_DATE('10/02/1984','DD/MM/YYYY'),
    TO_DATE('28/09/2011','DD/MM/YYYY'),'RUA Y, 44', 2345.52);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12356,        4,        'THIAGO        DE        ALMEIDA',
TO_DATE('10/03/1998','DD/MM/YYYY'),
    TO_DATE('24/10/2018','DD/MM/YYYY'),'RUA Y, 55', 1254.22);

-- DEPTO = 5
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12357,        5,        'LARISSSA        DE        CAMARGO',
TO_DATE('14/02/1997','DD/MM/YYYY'),
    TO_DATE('04/08/2015','DD/MM/YYYY'),'RUA V, 22', 1245.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12358,        5,        'ANTONIO        DE        CAMARGO',
TO_DATE('25/01/1985','DD/MM/YYYY'),
    TO_DATE('12/08/2016','DD/MM/YYYY'),'RUA V, 44', 2451.33);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12359,        5,        'JOSE        DE        CAMARGO',
TO_DATE('23/10/1998','DD/MM/YYYY'),
    TO_DATE('20/04/2017','DD/MM/YYYY'),'RUA V, 88', 6541.22);

-- POPULAR A TABELA DEPENDENTE. ESCOLHER 3 FUNCIONARIOS E
CADASTRAR 3 DEPENDENTES
-- PARA CADA FUNCIONARIO ESCOLHIDO
-- FUNCIONARIO = 12345
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 1, 'JOANINHA', TO_DATE('12/10/2010','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 2, 'JULINHA', TO_DATE('15/10/2012','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
(12345, 3, 'TONINHO', TO_DATE('22/10/2014','DD/MM/YYYY'));

-- FUNCIONARIO = 12356
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 1,'JUNINHO', TO_DATE('02/10/2011','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 2, 'ZEZINHO', TO_DATE('15/10/2013','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 3, 'MARCELINHO', TO_DATE('17/10/2015','DD/MM/YYYY'));

-- FUNCIONARIO = 12359
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 1,'MARIAZINHA', TO_DATE('11/06/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 2,'LUIZINHA', TO_DATE('21/06/2016','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 3,'CARMINHA', TO_DATE('18/06/2018','DD/MM/YYYY'));

-- POPULAR A TABELA PROJETO COM 5 PROJETOS
-- 3 PROJETOS SEM A DATA TÉRMINO
INSERT INTO T_SIP_PROJETO VALUES
(10001,   'PROJETO   1',   TO_DATE('11/10/2016','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10002,   'PROJETO   2',   TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
INSERT INTO T_SIP_PROJETO VALUES
(10003,   'PROJETO   3',   TO_DATE('28/04/2012','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10004,   'PROJETO   4',   TO_DATE('25/03/2017','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
(10005,   'PROJETO   5',   TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- POPULAR A TABELA IMPLANTACAO
-- PROJETO 1
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    1,    12345,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('15/09/2017','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001, 2, 12346, TO_DATE('11/10/2016','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    3,    12348,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('25/06/2018','DD/MM/YYYY'));
-- PROJETO 2
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    1,    12345,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    2,    12352,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('25/07/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    3,    12355,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
-- PROJETO 3
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 1, 12354, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 2, 12355, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 3, 12345, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
-- PROJETO 4
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 1, 12357, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
```

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
(10004, 2, 12358, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004,    3,    12354,    TO_DATE('25/03/2017','DD/MM/YYYY'),
TO_DATE('15/08/2018','DD/MM/YYYY'));
-- PROJETO 5
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    1,    12345,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    2,    12350,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    3,    12354,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- CONSULTAR TABELAS
SELECT * FROM T_SIP_DEPARTAMENTO;
SELECT * FROM T_SIP_FUNCIONARIO;
SELECT * FROM T_SIP_DEPENDENTE;
SELECT * FROM T_SIP_PROJETO;
SELECT * FROM T_SIP_IMPLANTACAO;

--EFETIVAR AS TRANSAÇÕES PENDENTES
COMMIT;
```

Código-fonte 3 – Script para popular as tabelas do banco de dados SIP
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.4 Pesquisa em múltiplas tabelas

Imagine se fosse necessário recuperar o nome do departamento em que o funcionário está trabalhando. O modelo abaixo, ilustra o relacionamento entre as tabelas “FUNCIONARIO” e “DEPARTAMENTO”.

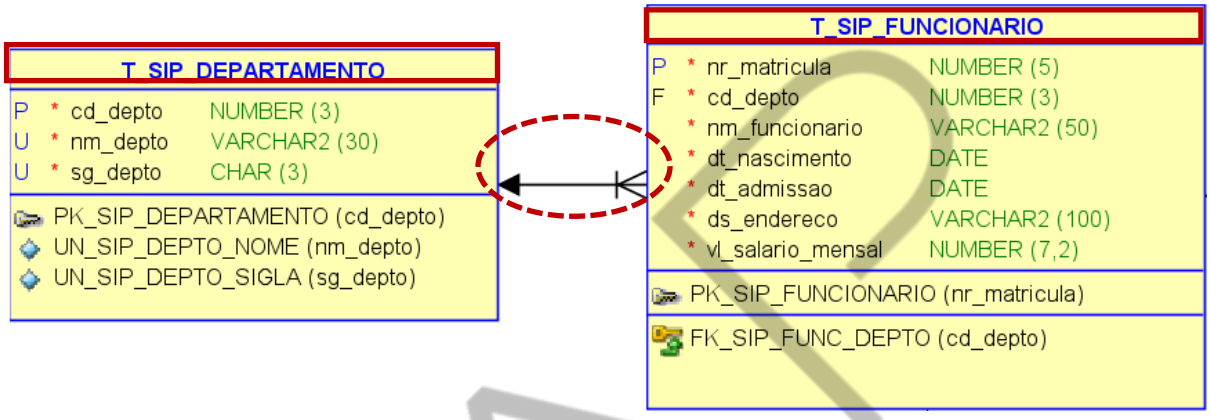


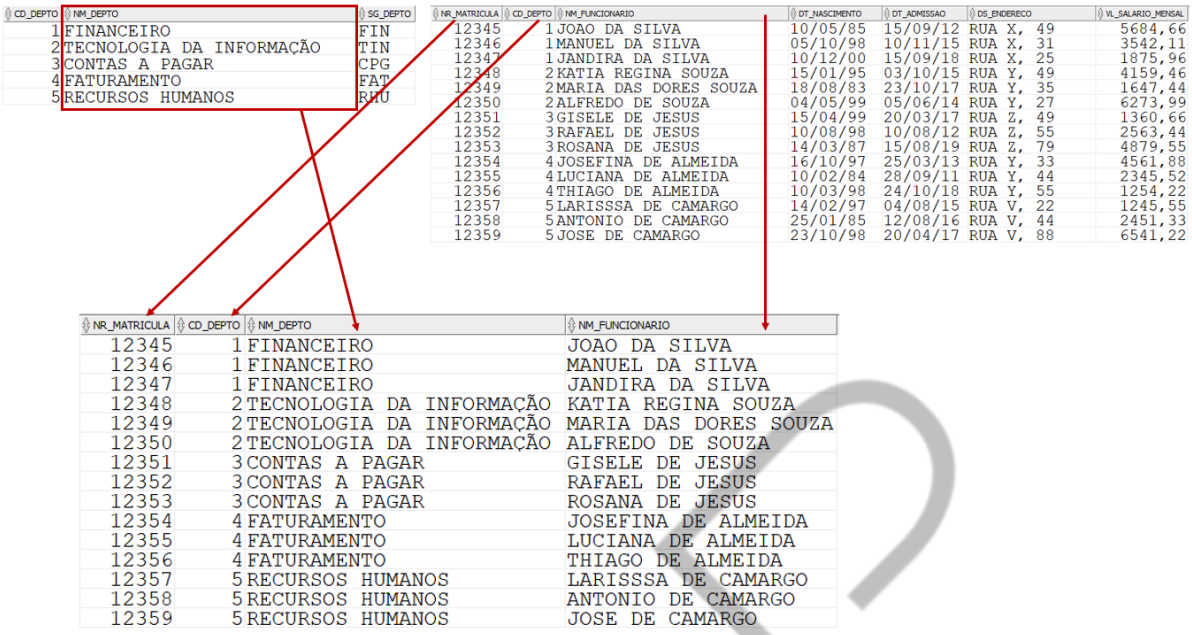
Figura 1 – Modelo E-R - Relacionamento
Fonte: Elaborado pela autora (2023)

O **nome do departamento** está armazenado na tabela **“DEPARTAMENTO”** que representa o cadastro dos departamentos. Existe uma associação entre estas duas tabelas, observamos a **chave estrangeira “CD_DEPTO”** na tabela **“FUNCIONARIO”.**

Para que possamos recuperar o nome do departamento para cada registro da tabela “FUNCIONARIO” precisaremos **combinar (JUNÇÃO) estas duas tabelas**.

A imagem abaixo ilustra a junção de tabelas. Para que possamos recuperar os dados: número da matrícula, código do departamento, nome do departamento e nome do funcionário. É necessário realizar a junção das tabelas, desta forma, conseguimos recuperar o nome do departamento em que o funcionário está alocado.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com



CD_DEPTO	NM_DEPTO	SG_DEPTO
1	FINANCEIRO	FIN
2	TECNOLOGIA DA INFORMAÇÃO	TIN
3	CONTAS A PAGAR	CPG
4	FATURAMENTO	FAT
5	RECURSOS HUMANOS	RHU

NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	4159,46
12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1647,44
12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	6273,99
12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

NR_MATRICULA	CD_DEPTO	NM_DEPTO	NM_FUNCIONARIO
12345	1	FINANCEIRO	JOAO DA SILVA
12346	1	FINANCEIRO	MANUEL DA SILVA
12347	1	FINANCEIRO	JANDIRA DA SILVA
12348	2	TECNOLOGIA DA INFORMAÇÃO	KATIA REGINA SOUZA
12349	2	TECNOLOGIA DA INFORMAÇÃO	MARIA DAS DORES SOUZA
12350	2	TECNOLOGIA DA INFORMAÇÃO	ALFREDO DE SOUZA
12351	3	CONTAS A PAGAR	GISELE DE JESUS
12352	3	CONTAS A PAGAR	RAFAEL DE JESUS
12353	3	CONTAS A PAGAR	ROSANA DE JESUS
12354	4	FATURAMENTO	JOSEFINA DE ALMEIDA
12355	4	FATURAMENTO	LUCIANA DE ALMEIDA
12356	4	FATURAMENTO	THIAGO DE ALMEIDA
12357	5	RECURSOS HUMANOS	LARISSSA DE CAMARGO
12358	5	RECURSOS HUMANOS	ANTONIO DE CAMARGO
12359	5	RECURSOS HUMANOS	JOSE DE CAMARGO

Figura 2 – Ilustração de junção de tabelas
Fonte: Elaborado pela autora (2023)

1.4.1 Qualificadores de Nome

Consistem no nome da tabela seguido de um ponto e o nome da coluna da tabela, conforme exemplo:

```
-- Exemplo de qualificador de nomes
SELECT   T_SIP_DEPARTAMENTO.NM_DEPTO
         Nome da tabela     Nome do campo
  FROM   T_SIP_DEPARTAMENTO ;
```

Código-fonte 4 – Exemplo de código-fonte SQL–comando SELECT com qualificador de nomes
Fonte: Elaborado pela autora (2023)

Colocar a identificação (qualificador) é opcional, porém é uma prática recomendada para facilitar o entendimento do comando. Quando estivermos recuperando colunas com mesmo nome, em tabelas diferentes, se faz necessário informar a tabela que se deseja recuperar a informação, e neste caso, a utilização do qualificador se torna obrigatória.

Exemplo de instrução SQL com qualificador de nomes:

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
-- Exemplo de consulta com qualificador de nomes
SELECT   T_SIP_FUNCIONARIO.NR_MATRICULA ,
         T_SIP_FUNCIONARIO.CD_DEPTO     ,
         T_SIP_FUNCIONARIO.DT_ADMISSAO  ,
         T_SIP_FUNCIONARIO.VL_SALARIO_MENSAL
  FROM   T_SIP_FUNCIONARIO ;
```

Código-fonte 5 – Exemplo de código-fonte SQL-Comando SELECT com qualificador de nomes
Fonte: Elaborado pela autora (2023)

1.4.2 ALIAS – Apelido para tabelas

Quando consultamos duas ou mais tabelas, os nomes das tabelas são usados nas cláusulas SELECT e WHERE. Com isso, observamos a repetição de nomes das tabelas, ou seja, temos uma digitação redundante.

Para facilitar a escrita e manutenção dos comandos, podemos definir apelidos para o nome da tabela. Fazemos isto na cláusula FROM, assim, é possível usar esses apelidos para referenciar as tabelas em outro ponto da consulta, como nas cláusulas SELECT e WHERE.

Exemplo de consulta a duas tabelas sem utilização de ALIAS.

```
-- Exemplo de consulta a duas tabelas sem ALIAS
SELECT T_SIP_FUNCIONARIO.NM_FUNCIONARIO ,
       T_SIP_FUNCIONARIO.CD_DEPTO       ,
       T_SIP_DEPARTAMENTO.NM_DEPTO
  FROM T_SIP_DEPARTAMENTO INNER JOIN T_SIP_FUNCIONARIO
    ON T_SIP_DEPARTAMENTO.CD_DEPTO = T_SIP_FUNCIONARIO.CD_DEPTO;
```

Código-fonte 6 – Exemplo de código-fonte SQL – comando SELECT sem ALIAS de tabela
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando SELECT:

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

NM_FUNCIONARIO	CD_DEPTO	NM_DEPTO
JOAO DA SILVA	1	FINANCEIRO
MANUEL DA SILVA	1	FINANCEIRO
JANDIRA DA SILVA	1	FINANCEIRO
KATIA REGINA SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
MARIA DAS DORES SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
ALFREDO DE SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
GISELE DE JESUS	3	CONTAS A PAGAR
RAFAEL DE JESUS	3	CONTAS A PAGAR
ROSANA DE JESUS	3	CONTAS A PAGAR
JOSEFINA DE ALMEIDA	4	FATURAMENTO
LUCIANA DE ALMEIDA	4	FATURAMENTO
THIAGO DE ALMEIDA	4	FATURAMENTO
LARISSSA DE CAMARGO	5	RECURSOS HUMANOS
ANTONIO DE CAMARGO	5	RECURSOS HUMANOS
JOSE DE CAMARGO	5	RECURSOS HUMANOS

Figura 3 – Resultado da consulta às tabelas DEPARTAMENTO e FUNCIONARIO sem ALIAS
Fonte: Elaborado pela autora (2023).

Exemplo de consulta a duas tabelas com utilização de ALIAS.

```
-- Exemplo de consulta a duas tabelas com ALIAS
SELECT F.NM_FUNCIONARIO ,
       F.CD_DEPTO       ,
       D.NM_DEPTO
  FROM T_SIP_DEPARTAMENTO D INNER JOIN T_SIP_FUNCIONARIO F
       ON (D.CD_DEPTO = F.CD_DEPTO);
```

Código-fonte 7 – Exemplo de código-fonte SQL – comando SELECT com ALIAS de tabela
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando SELECT:

FIAP

NM_FUNCIONARIO	CD_DEPTO	NM_DEPTO
JOAO DA SILVA	1	FINANCEIRO
MANUEL DA SILVA	1	FINANCEIRO
JANDIRA DA SILVA	1	FINANCEIRO
KATIA REGINA SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
MARIA DAS DORES SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
ALFREDO DE SOUZA	2	TECNOLOGIA DA INFORMAÇÃO
GISELE DE JESUS	3	CONTAS A PAGAR
RAFAEL DE JESUS	3	CONTAS A PAGAR
ROSANA DE JESUS	3	CONTAS A PAGAR
JOSEFINA DE ALMEIDA	4	FATURAMENTO
LUCIANA DE ALMEIDA	4	FATURAMENTO
THIAGO DE ALMEIDA	4	FATURAMENTO
LARISSSA DE CAMARGO	5	RECURSOS HUMANOS
ANTONIO DE CAMARGO	5	RECURSOS HUMANOS
JOSE DE CAMARGO	5	RECURSOS HUMANOS

Figura 4 – Resultado da consulta às tabelas DEPARTAMENTO e FUNCIONARIO com ALIAS
Fonte: Elaborado pela autora (2023)

1.4.3 Cross Join – Junção Cruzada

A junção cruzada é também conhecida por **Produto Cartesiano**.

Ocorrerá um produto cartesiano sempre que:

- Não houver uma condição para a união (ausência da cláusula WHERE).
- Condição de união entre as tabelas inválida (cláusula WHERE incorreta).
- Todas as linhas da primeira tabela estiverem unidas a todas as linhas da segunda tabela.

Para evitar um produto cartesiano, utilize uma condição válida para junção na cláusula WHERE.

A ilustração abaixo exemplifica um CROSS JOIN. Todas os registros (linhas) da tabela “COMPANY” são combinados com registros (linhas) da tabela “FOODS”, gerando o produto cartesiano.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

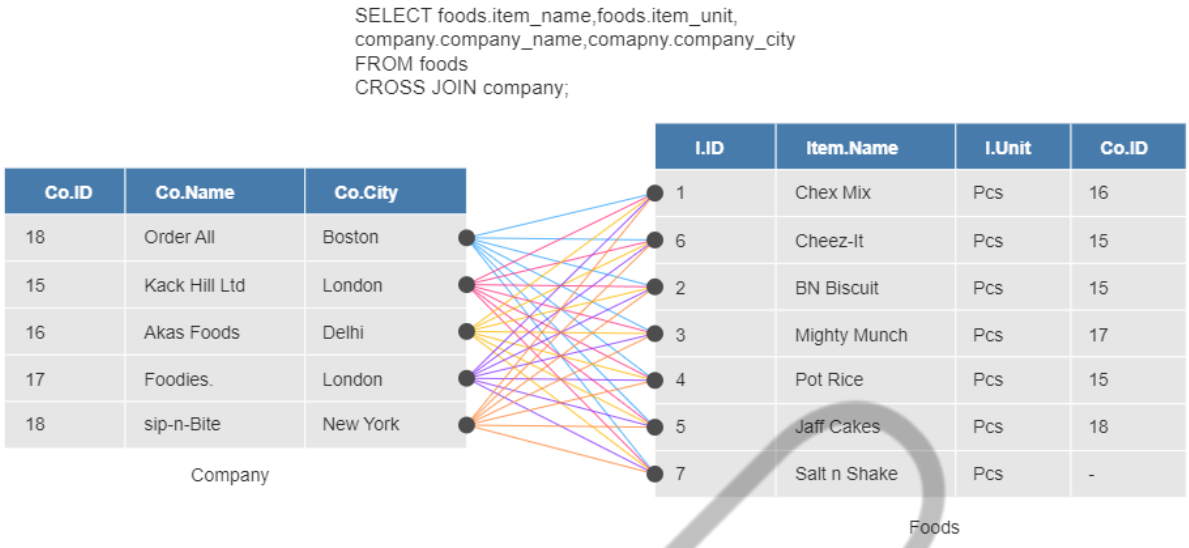


Figura 5 – Cross Join
Fonte: Google Imagens (2017)

Exemplo de junção cruzada (CROSS JOIN) entre as tabelas: “DEPARTAMENTO” e “FUNCIONARIO”.

A tabela “DEPARTAMENTO” possui cinco registros (linhas).

	CD_DEPTO	NM_DEPTO	SG_DEPTO
1	1	FINANCEIRO	FIN
2	2	TECNOLOGIA DA INFORMAÇÃO	TIN
3	3	CONTAS A PAGAR	CPG
4	4	FATURAMENTO	FAT
5	5	RECURSOS HUMANOS	RHU

Figura 6 – Conteúdo da tabela DEPARTAMENTO
Fonte: Elaborado pela autora (2023)

A tabela “FUNCIONARIO” possui quinze registros (linhas).

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
2	12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
3	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
4	12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	4159,46
5	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1647,44
6	12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	6273,99
7	12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
8	12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
9	12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
10	12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
11	12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
13	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
14	12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
15	12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 7 – Conteúdo da tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

O comando SELECT retorna à combinação (junção) das tabelas “DEPARTAMENTO” e “FUNCIONARIO”.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
-- UTILIZANDO O PADRÃO ANSI SQL/99
-- CROSS JOIN
SELECT *
  FROM T_SIP_DEPARTAMENTO
       CROSS JOIN T_SIP_FUNCIONARIO;
```

Código-fonte 8 – Exemplo de código-fonte SQL – CROSS JOIN
Fonte: Elaborado pela autora (2017)

É boa prática utilizar a sintaxe conforme padrão ANSI – SQL/99.
1) Permite migrar a aplicação para outro SGBD, sem alteração de código.
2) Ligações entre colunas são simplificadas, pois ficam fora da cláusula WHERE, separadas das condições de filtro.
Esta característica, facilita manutenções futuras.
Em relação à performance, não há diferenças entre os padrões ANSI e Oracle, a maioria das instruções SQL, quando executadas, é convertida internamente para o dialeto Oracle. A exceção é o FULL OUTER JOIN, por ter uma sintaxe simplificada (melhor performance e menor complexidade) no padrão ANSI.

Resultado da execução do comando “SELECT” com CROSS JOIN.

Resultado da Consulta

Todas as Linhas Extraídas: 75 em 0,03 segundos

	CD_DEPTO	NM_DEPTO	SG_DEPTO	NR_MATRICULA	CD_DEPTO_1	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	1	FINANCEIRO	FIN	12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
2	1	FINANCEIRO	FIN	12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
3	1	FINANCEIRO	FIN	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
4	1	FINANCEIRO	FIN	12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	4159,46
5	1	FINANCEIRO	FIN	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1647,44
6	1	FINANCEIRO	FIN	12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	6273,99
7	1	FINANCEIRO	FIN	12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
8	1	FINANCEIRO	FIN	12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
9	1	FINANCEIRO	FIN	12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
10	1	FINANCEIRO	FIN	12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
11	1	FINANCEIRO	FIN	12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12	1	FINANCEIRO	FIN	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
13	1	FINANCEIRO	FIN	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
14	1	FINANCEIRO	FIN	12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
15	1	FINANCEIRO	FIN	12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22
16	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
17	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
18	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
19	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	4159,46
20	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1647,44
21	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	6273,99
22	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
23	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
24	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
25	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
26	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
27	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
28	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
29	2	TECNOLOGIA DA INFORMAÇÃO	TIN	12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33

Figura 8 – Resultado do CROSS JOIN
Fonte: Elaborado pela autora (2023)

O que ocorreu no exemplo foi que as linhas da primeira tabela foram combinadas com as linhas da segunda tabela, demonstrando um resultado na maior parte das vezes indesejado.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Veja que a tabela “DEPARTAMENTO” possuía 5 linhas e a tabela “FUNCIONARIO” possuía 15 linhas, o resultado foram (5X15=75) 75 linhas exibidas como resultado da nossa busca.

Percebemos então a necessidade de colocar a condição após a cláusula WHERE associando a chave primária e estrangeira das tabelas. Utilize o modelo de dados e visualize o relacionamento para facilitar a criação de sua consulta SQL.

1.4.4 Inner Join – Junção Interna

A junção do tipo **INNER JOIN** segue a **padronização ANSI** e implica, por sua vez, em criar a **junção interna entre duas tabelas**. Esse tipo de junção surgiu a partir da padronização ANSI 99.

Nesse tipo de junção, caso sejam unidas duas tabelas, serão exibidos todos os dados existentes nas duas tabelas envolvidas na consulta, em que encontrarmos correspondência entre as chaves primária e estrangeira.

Denomina-se união regular aquelas uniões que têm a cláusula WHERE indicando a chave primária à estrangeira das tabelas afetadas pelo comando SELECT.

Podemos ilustrar a junção interna através da intersecção entre dois conjuntos.

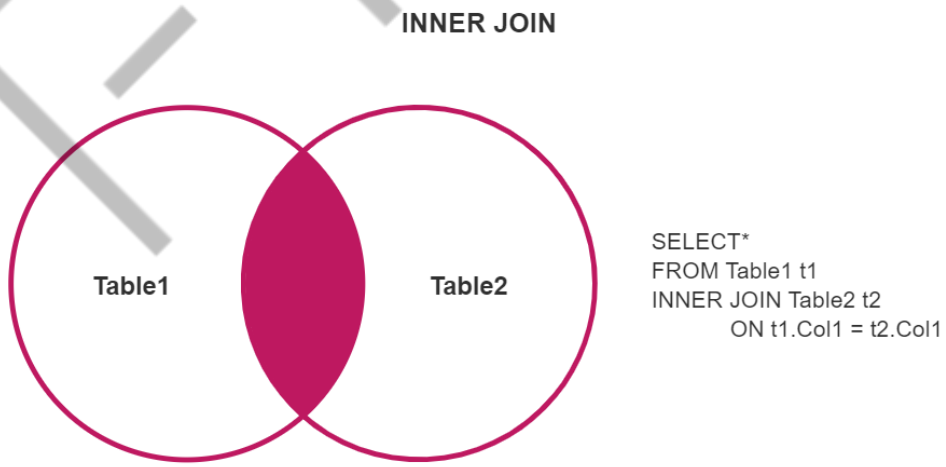


Figura 9 – INNER JOIN
Fonte: Google imagens (2017)

O comando SELECT retornará à combinação (junção) das tabelas “DEPARTAMENTO” e “FUNCIONARIO” a partir de um INNER JOIN.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

A condição de junção é estabelecida na cláusula ON, em que comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção entre as tabelas.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos, ou seja, apenas os registros (linhas) relacionados entre as duas tabelas. Em outras palavras, serão recuperados todos os funcionários associados aos seus respectivos departamentos, ordenados por nome do departamento.

```
-- EXEMPLO - INNER JOIN – Padrão SQL/99
  SELECT F.NR_MATRICULA, F.CD_DEPTO,
         D.NM_DEPTO, F.NM_FUNCIONARIO
    FROM T_SIP_DEPARTAMENTO D INNER JOIN T_SIP_FUNCIONARIO  F
         ON ( D.CD_DEPTO = F.CD_DEPTO )
ORDER BY D.NM_DEPTO;
```

Código-fonte 9 – Exemplo de código-fonte SQL – INNER JOIN
Fonte: Elaborado pela autora (2023)

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Resultado da execução do comando “SELECT” com INNER JOIN.

NR_MATRICULA	CD_DEPTO	NM_DEPTO	NM_FUNCIONARIO
12352	3	CONTAS A PAGAR	RAFAEL DE JESUS
12351	3	CONTAS A PAGAR	GISELE DE JESUS
12353	3	CONTAS A PAGAR	ROSANA DE JESUS
12356	4	FATURAMENTO	THIAGO DE ALMEIDA
12355	4	FATURAMENTO	LUCIANA DE ALMEIDA
12354	4	FATURAMENTO	JOSEFINA DE ALMEIDA
12347	1	FINANCEIRO	JANDIRA DA SILVA
12345	1	FINANCEIRO	JOAO DA SILVA
12346	1	FINANCEIRO	MANUEL DA SILVA
12359	5	RECURSOS HUMANOS	JOSE DE CAMARGO
12357	5	RECURSOS HUMANOS	LARISSSA DE CAMARGO
12358	5	RECURSOS HUMANOS	ANTONIO DE CAMARGO
12349	2	TECNOLOGIA DA INFORMAÇÃO	MARIA DAS DORES SOUZA
12348	2	TECNOLOGIA DA INFORMAÇÃO	KATIA REGINA SOUZA
12350	2	TECNOLOGIA DA INFORMAÇÃO	ALFREDO DE SOUZA

Figura 10 – Resultado do INNER JOIN
Fonte: Elaborado pela autora (2023)

1.4.5 Inner Join com cláusula Using – Junção Interna

Caso as colunas tenham o mesmo nome, mas tipos de dados diferentes, podemos modificar a cláusula **INNER JOIN** com a **cláusula USING** para especificar as colunas que devem ser usadas em uma junção. **Não utilize apelido ou nome de tabelas nas colunas** em que a referência for feita (chave primária/chave estrangeira).

O comando SELECT retornará à combinação (junção) das tabelas “DEPARTAMENTO” e “FUNCIONARIO” a partir de um INNER JOIN.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos, ou seja, apenas os registros (linhas) relacionados entre as duas tabelas. Em outras palavras, serão recuperados todos os funcionários associados aos seus respectivos departamentos, ordenados por nome do departamento.

```
-- EXEMPLO - INNER JOIN com a cláusula USING
  SELECT F.NR_MATRICULA,
         CD_DEPTO,
         D.NM_DEPTO, F.NM_FUNCIONARIO
    FROM T_SIP_DEPARTAMENTO D INNER JOIN T_SIP_FUNCIONARIO  F
         USING ( CD_DEPTO )
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
ORDER BY NM_DEPTO;
```

Código-fonte 10 – Exemplo de código-fonte SQL – INNER JOIN com USING
Fonte: Elaborado pela autora (2023)

Nota: Não aplique o qualificador na coluna “CD_DEPTO”, pois acarretará um erro. Quando utilizamos INNER JOIN com a cláusula USING, não podemos utilizar qualificadores para a colunas chave primária/estrangeira.

Resultado da execução do comando “SELECT” com INNER JOIN e cláusula USING.

NR_MATRICULA	CD_DEPTO	NM_DEPTO	NM_FUNCIONARIO
12352	3	CONTAS A PAGAR	RAFAEL DE JESUS
12351	3	CONTAS A PAGAR	GISELE DE JESUS
12353	3	CONTAS A PAGAR	ROSANA DE JESUS
12356	4	FATURAMENTO	THIAGO DE ALMEIDA
12355	4	FATURAMENTO	LUCIANA DE ALMEIDA
12354	4	FATURAMENTO	JOSEFINA DE ALMEIDA
12347	1	FINANCEIRO	JANDIRA DA SILVA
12345	1	FINANCEIRO	JOAO DA SILVA
12346	1	FINANCEIRO	MANUEL DA SILVA
12359	5	RECURSOS HUMANOS	JOSE DE CAMARGO
12357	5	RECURSOS HUMANOS	LARISSSA DE CAMARGO
12358	5	RECURSOS HUMANOS	ANTONIO DE CAMARGO
12349	2	TECNOLOGIA DA INFORMAÇÃO	MARIA DAS DORES SOUZA
12348	2	TECNOLOGIA DA INFORMAÇÃO	KATIA REGINA SOUZA
12350	2	TECNOLOGIA DA INFORMAÇÃO	ALFREDO DE SOUZA

Figura 11 – Resultado do INNER JOIN com USING
Fonte: Elaborado pela autora (2023)

1.4.6 Inner Join com duas ou mais tabelas – Junção Interna

Podemos combinar duas ou mais tabelas, de acordo com a necessidade de negócio, que nos determina o que será necessário recuperar e apresentar ao usuário.

Podemos recuperar os dados de um funcionário mostrando os projetos em que ele está atuando, por exemplo.

O comando SELECT retornará à combinação (junção) das tabelas “PROJETO”, “IMPLANTACAO” e “FUNCIONARIO” a partir de um INNER JOIN.

A condição de junção é estabelecida na cláusula ON, na qual comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção entre as tabelas.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Como no modelo E-R, os relacionamentos são estabelecidos aos pares, na comparação, através da cláusula ON, confrontamos a chave primária a chave estrangeira, aos pares, respeitando os relacionamentos.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos, ou seja, apenas os registros (linhas) relacionados entre as tabelas, identificadas na cláusula FROM. Em outras palavras, serão recuperados todos os funcionários associados aos projetos em que estão atuando (implantando), ordenados por nome do funcionário.

```
-- EXEMPLO - INNER JOIN – Padrão SQL/99
  SELECT F.NR_MATRICULA, F.NM_FUNCIONARIO,
         P.NM_PROJETO,
         I.DT_ENTRADA,I.DT_SAIDA
    FROM T_SIP_PROJETO P INNER JOIN T_SIP_IMPLANTACAO I
         ON ( P.CD_PROJETO = I.CD_PROJETO )
         INNER JOIN  T_SIP_FUNCIONARIO  F
         ON ( F.NR_MATRICULA = I.NR_MATRICULA )
ORDER BY F.NM_FUNCIONARIO ;
```

Código-fonte 11 – Exemplo de código-fonte SQL – INNER JOIN – Várias tabelas
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com INNER JOIN, combinando várias tabelas.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

NR_MATRICULA	NM_FUNCIONARIO	NM_PROJETO	DT_ENTRADA	DT_SAIDA
12350	ALFREDO DE SOUZA	PROJETO 5	28/09/14	15/09/18
12358	ANTONIO DE CAMARGO	PROJETO 4	25/03/17	(null)
12345	JOAO DA SILVA	PROJETO 3	28/04/13	(null)
12345	JOAO DA SILVA	PROJETO 5	28/09/14	15/09/18
12345	JOAO DA SILVA	PROJETO 1	11/10/16	15/09/17
12345	JOAO DA SILVA	PROJETO 2	25/10/13	15/04/14
12354	JOSEFINA DE ALMEIDA	PROJETO 4	25/03/17	15/08/18
12354	JOSEFINA DE ALMEIDA	PROJETO 5	28/09/14	15/09/18
12354	JOSEFINA DE ALMEIDA	PROJETO 3	28/04/13	(null)
12348	KATIA REGINA SOUZA	PROJETO 1	11/10/16	25/06/18
12357	LARISSSA DE CAMARGO	PROJETO 4	25/03/17	(null)
12355	LUCIANA DE ALMEIDA	PROJETO 2	25/10/13	15/04/15
12355	LUCIANA DE ALMEIDA	PROJETO 3	28/04/13	(null)
12346	MANUEL DA SILVA	PROJETO 1	11/10/16	(null)
12352	RAFAEL DE JESUS	PROJETO 2	25/10/13	25/07/14

Figura 12 – Resultado do INNER JOIN com várias tabelas
Fonte: Elaborado pela autora (2023)

Exemplo da sintaxe empregando a cláusula USING, usado quando as colunas têm o mesmo nome.

```
-- EXEMPLO COM INNER JOIN - PADRÃO SQL/99 -
-- CLÁUSULA USING
  SELECT NR_MATRICULA, F.NM_FUNCIONARIO,
         P.NM_PROJETO,
         I.DT_ENTRADA,I.DT_SAIDA
    FROM T_SIP_PROJETO P INNER JOIN T_SIP_IMPLANTACAO I
         USING ( CD_PROJETO )
         INNER JOIN  T_SIP_FUNCIONARIO  F
         USING ( NR_MATRICULA )
ORDER BY F.NM_FUNCIONARIO ;
```

Código-fonte 12 – Exemplo de código-fonte SQL – INNER JOIN com USING
Fonte: Elaborado pela autora (2023)

Nota: Não aplique o qualificador na coluna “NR_MATRICULA”, pois acarretará um erro. Quando utilizamos INNER JOIN com a cláusula USING, não podemos utilizar qualificadores para a colunas chave primária/estrangeira.

Dica: No padrão SQL a ordem em que colocamos as tabelas na cláusula FROM determina quais tabelas serão pesquisadas primeiro. Logo, se colocarmos as tabelas menores primeiro, a busca ficará mais rápida. Portanto, deixe as tabelas maiores, sempre que possível, para o final da cláusula.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.5 Junções Externas

Junção externa é utilizada para recuperar as linhas de um EQUIJOIN, sendo que em alguma das tabelas pode não existir registros (linhas) correspondentes. A junção externa pode ser esquerda, direita ou completa.

Define-se união externa como aquela que inclui linhas no resultado da busca, mesmo que não haja relação entre as duas tabelas que estão sendo combinadas (unidas). Onde não há informação, será recuperado NULL.

Existem 3 formas de realizar a união externa:

- **Left Outer Join**: a junção externa à esquerda recupera todas as linhas do EQUIJOIN, além das que não possuem correspondentes na tabela à esquerda da operação.
- **Right Outer Join**: a junção externa à direita recupera todas as linhas do EQUIJOIN, além das que não possuem correspondentes na tabela à direita da operação.
- **Full Outer Join**: a junção completa recupera todas as linhas do EQUIJOIN, além das que não possuem correspondentes na tabela à direita e a esquerda da operação.

1.5.1 Left Outer Join – Junções Externas

A junção do tipo **LEFT OUTER JOIN** segue a **padronização ANSI** e implica, por sua vez, em criar a **junção externa entre duas tabelas**. É um tipo de junção externa. Esse tipo de junção surgiu a partir da padronização ANSI 99.

Nesse tipo de junção, todas as linhas da tabela à esquerda serão recuperadas, independentemente da existência de ocorrências relacionadas na tabela da direita, ou seja, correspondência entre a chave primária à chave estrangeira.

Preserva as linhas sem correspondência da primeira tabela (esquerda), juntando-as com uma linha nula na forma da segunda tabela (direita).

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Denomina-se união regular (EQUIJOIN) aquelas uniões que têm a cláusula WHERE indicando a chave primária à estrangeira das tabelas afetadas pelo comando SELECT.

Podemos ilustrar a junção externa, através dos exemplos abaixo.

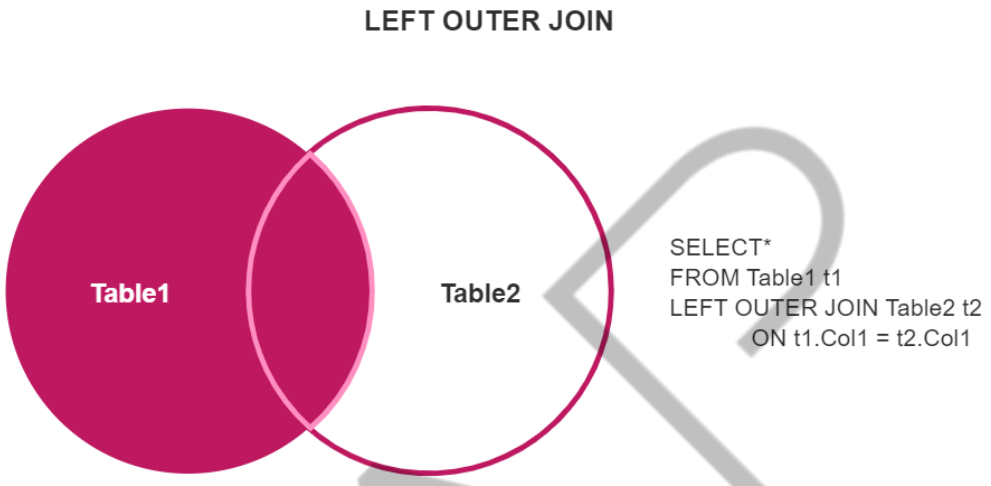


Figura 13 – LEFT OUTER JOIN
Fonte: Google imagens (2017)

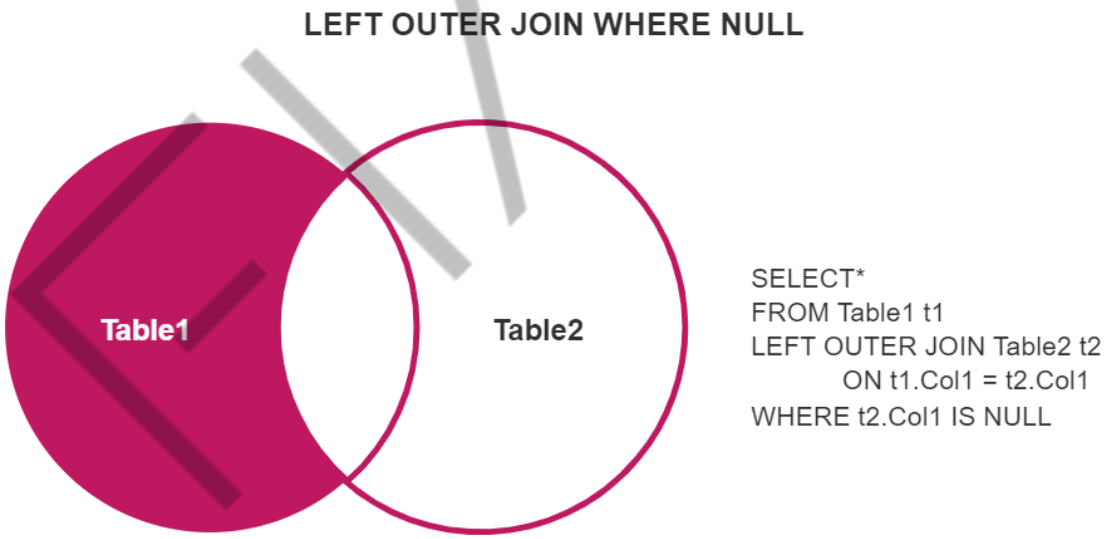


Figura 14 – LEFT OUTER JOIN com WHERE NULL
Fonte: Google imagens (2017)

O comando SELECT retornará à combinação (junção) das tabelas à esquerda “PROJETO” e “IMPLANTACAO” a partir de um LEFT OUTER JOIN.

A condição de junção é estabelecida na cláusula ON, na qual comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção ou não entre as tabelas.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos ou não, ou seja, são recuperados os registros (linhas) relacionados entre as duas tabelas e aqueles que não possuem associações. Em outras palavras, serão recuperados todos os projetos que tenham ou não funcionários participando de implantações. No caso dos projetos que não têm funcionários atuando em implantações no momento da recuperação, esses dados serão preenchidos com NULL.

```
-- EXEMPLO LEFT OUTER JOIN - PADRÃO SQL/99
SELECT    P.CD_PROJETO, P.NM_PROJETO, P.DT_INICIO,
                I.NR_MATRICULA, I.DT_ENTRADA
  FROM    T_SIP_PROJETO P LEFT OUTER JOIN T_SIP_IMPLANTACAO I
          ON ( P.CD_PROJETO = I.CD_PROJETO );
```

Código-fonte 13 – Exemplo de código-fonte SQL – LEFT OUTER JOIN
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com LEFT OUTER JOIN.

CD_PROJETO	NM_PROJETO	DT_INICIO	NR_MATRICULA	DT_ENTRADA
10001	PROJETO 1	11/10/16	12345	11/10/16
10001	PROJETO 1	11/10/16	12346	11/10/16
10001	PROJETO 1	11/10/16	12348	11/10/16
10002	PROJETO 2	25/10/13	12345	25/10/13
10002	PROJETO 2	25/10/13	12352	25/10/13
10002	PROJETO 2	25/10/13	12355	25/10/13
10003	PROJETO 3	28/04/12	12354	28/04/13
10003	PROJETO 3	28/04/12	12355	28/04/13
10003	PROJETO 3	28/04/12	12345	28/04/13
10004	PROJETO 4	25/03/17	12357	25/03/17
10004	PROJETO 4	25/03/17	12358	25/03/17
10004	PROJETO 4	25/03/17	12354	25/03/17
10005	PROJETO 5	28/09/14	12345	28/09/14
10005	PROJETO 5	28/09/14	12350	28/09/14
10005	PROJETO 5	28/09/14	12354	28/09/14
10006	PROJETO 6	05/04/20	(null)	(null)

Figura 15 – Resultado do LEFT OUTER JOIN
Fonte: Elaborado pela autora (2023)

Podemos recuperar somente os projetos que não têm funcionários atuando em implantações. Para isso, utilizaríamos o comando “SELECT” com LEFT OUTER JOIN

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

e a condição, em que serão recuperadas apenas as combinações que resultaram em NULOS.

```
-- EXEMPLO COM LEFT OUTER JOIN - PADRÃO SQL/99 -
-- COM VALIDAÇÃO DOS RESULTADOS NULOS
SELECT    P.CD_PROJETO, P.NM_PROJETO, P.DT_INICIO,
          I.NR_MATRICULA, I.DT_ENTRADA
  FROM    T_SIP_PROJETO P LEFT OUTER JOIN T_SIP_IMPLANTACAO I
          ON ( P.CD_PROJETO = I.CD_PROJETO )
 WHERE    I.CD_PROJETO IS NULL ;
```

Código-fonte 14 – Exemplo de código-fonte SQL – LEFT OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com LEFT OUTER JOIN e recuperação apenas de NULOS.

CD_PROJETO	NM_PROJETO	DT_INICIO	NR_MATRICULA	DT_ENTRADA
10006	PROJETO 6	05/04/20	(null)	(null)

Figura 16 – Resultado do LEFT OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

Exemplo da sintaxe aplicando a cláusula USING, usada quando as colunas têm o mesmo nome.

```
-- EXEMPLO LEFT OUTER JOIN - PADRÃO SQL/99
-- CLÁUSULA USING
SELECT    CD_PROJETO, P.NM_PROJETO, P.DT_INICIO,
          I.NR_MATRICULA, I.DT_ENTRADA
  FROM    T_SIP_PROJETO P LEFT OUTER JOIN T_SIP_IMPLANTACAO I
          USING ( CD_PROJETO );
```

Código-fonte 15 – Exemplo de código-fonte SQL – LEFT OUTER JOIN com USING
Fonte: Elaborado pela autora (2023)

Nota: Não aplique o qualificador na coluna “CD_PROJETO”, pois acarretará um erro. Quando utilizamos LEFT OUTER JOIN com a cláusula USING, não podemos utilizar qualificadores para a colunas chave primária/estrangeira.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.5.2 Right Outer Join – Junções Externas

A junção do tipo **RIGHT OUTER JOIN** segue a **padronização ANSI** e implica, por sua vez, em criar a **junção externa entre duas tabelas,** é um tipo de junção externa. Esse tipo de junção surgiu a partir da padronização ANSI 99.

Nesse tipo de junção, todas as linhas da tabela à direita serão recuperadas, independentemente da existência de ocorrências relacionadas na tabela da esquerda, ou seja, correspondência entre a chave primária a chave estrangeira.

Preserva as linhas sem correspondência da primeira tabela (direita), juntando-as com uma linha nula na forma da segunda tabela (esquerda).

Denomina-se união regular (EQUIJOIN) aquelas as uniões que têm a cláusula WHERE indicando a chave primária a estrangeira das tabelas afetadas pelo comando SELECT.

Podemos ilustrar a junção externa, através dos exemplos abaixo.

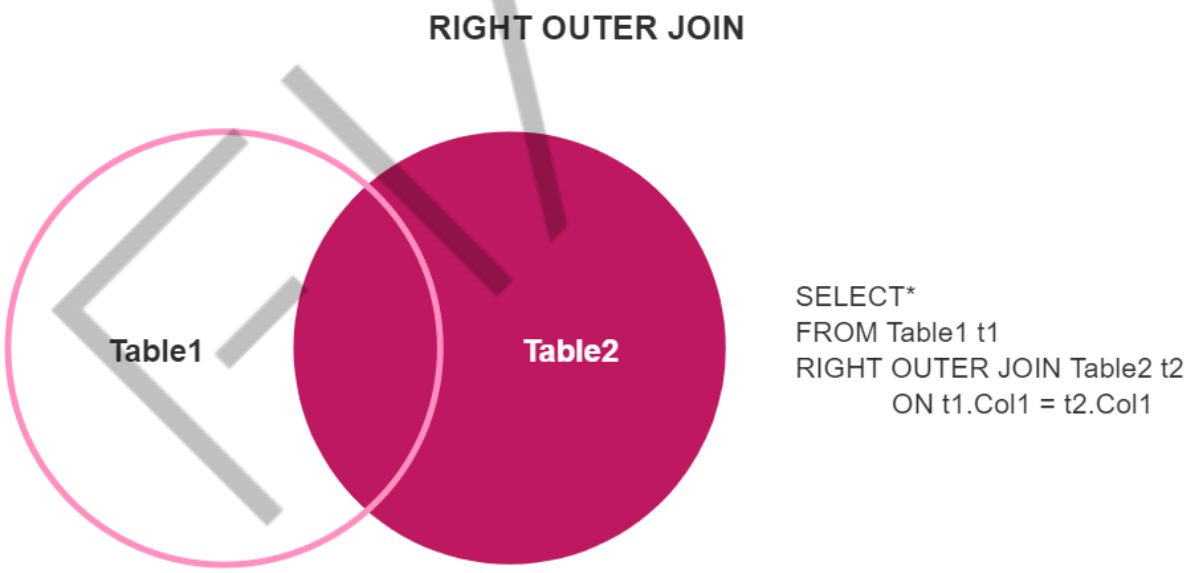


Figura 17 – RIGHT OUTER JOIN
Fonte: Google imagens (2017)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

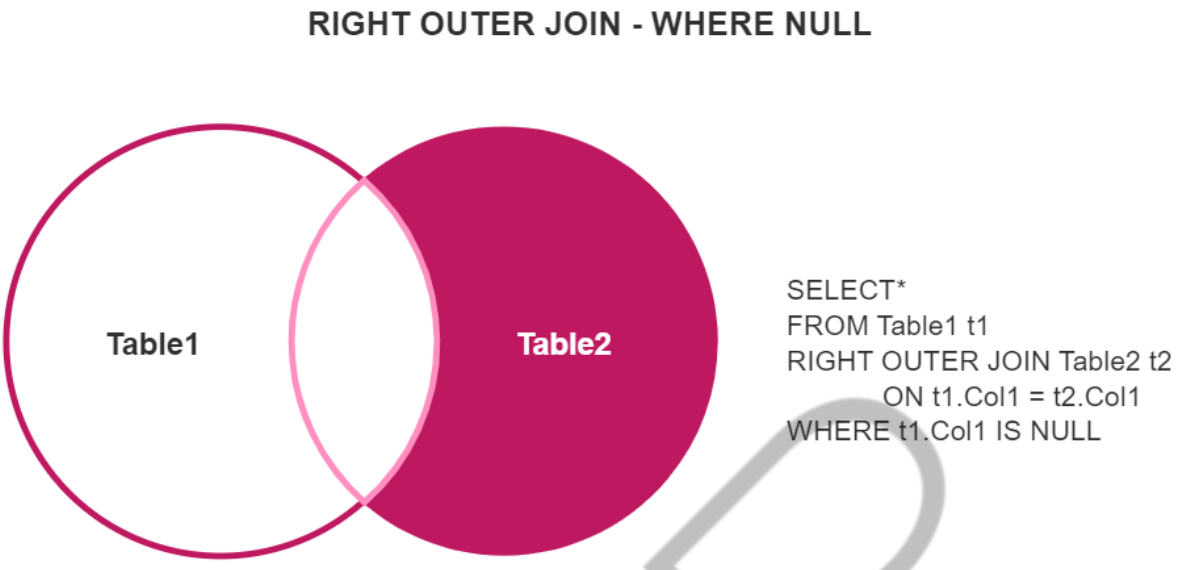


Figura 18 – RIGHT OUTER JOIN com WHERE NULL
Fonte: Google imagens (2017)

O comando SELECT retornará à combinação (junção) das tabelas à esquerda "IMPLANTACAO" e "FUNCIONARIO" a partir de um RIGHT OUTER JOIN.

A condição de junção é estabelecida na cláusula ON, na qual comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção ou não entre as tabelas.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos ou não, ou seja, são recuperados os registros (linhas) relacionados entre as duas tabelas e aqueles que não possuem associações. Em outras palavras, serão recuperados todos os funcionários que atuem ou não em projetos. No caso dos funcionários não atuantes em implantações, no momento da recuperação, esses dados serão preenchidos com NULL.

```
-- EXEMPLO RIGHT OUTER JOIN - PADRÃO SQL/99
SELECT F.NR_MATRICULA,
       I.CD_PROJETO, I.DT_ENTRADA,I.DT_SAIDA
FROM  T_SIP_IMPLANTACAO I RIGHT OUTER JOIN T_SIP_FUNCIONARIO F
         ON ( F.NR_MATRICULA = I.NR_MATRICULA );
```

Código-fonte 16 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando "SELECT" com RIGHT OUTER JOIN.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

NR_MATRICULA	CD_PROJETO	DT_ENTRADA	DT_SAIDA
12345	10001	11/10/16	15/09/17
12346	10001	11/10/16	(null)
12348	10001	11/10/16	25/06/18
12345	10002	25/10/13	15/04/14
12352	10002	25/10/13	25/07/14
12355	10002	25/10/13	15/04/15
12354	10003	28/04/13	(null)
12355	10003	28/04/13	(null)
12345	10003	28/04/13	(null)
12357	10004	25/03/17	(null)
12358	10004	25/03/17	(null)
12354	10004	25/03/17	15/08/18
12345	10005	28/09/14	15/09/18
12350	10005	28/09/14	15/09/18
12354	10005	28/09/14	15/09/18
12356	(null)	(null)	(null)
12353	(null)	(null)	(null)
12359	(null)	(null)	(null)
12349	(null)	(null)	(null)
12347	(null)	(null)	(null)
12351	(null)	(null)	(null)

Figura 19 – Resultado do RIGHT OUTER JOIN
Fonte: Elaborado pela autora (2023)

Podemos recuperar somente os funcionários que não atuam em implantações. Para isso, utilizaríamos o comando “SELECT” com RIGHT OUTER JOIN e a condição, na qual serão recuperadas apenas as combinações que resultaram em NULOS.

```
-- EXEMPLO COM RIGHT OUTER JOIN - PADRÃO SQL/99 -
-- COM VALIDAÇÃO DOS RESULTADOS NULOS
SELECT   F.NR_MATRICULA,
         I.CD_PROJETO, I.DT_ENTRADA,I.DT_SAIDA
FROM  T_SIP_IMPLANTACAO I RIGHT OUTER JOIN T_SIP_FUNCIONARIO F
         ON ( F.NR_MATRICULA = I.NR_MATRICULA )
 WHERE   I.NR_MATRICULA IS NULL ;
```

Código-fonte 17 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com RIGHT OUTER JOIN e recuperação apenas de NULOS.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

NR_MATRICULA	CD_PROJETO	DT_ENTRADA	DT_SAIDA
12356	(null)	(null)	(null)
12353	(null)	(null)	(null)
12359	(null)	(null)	(null)
12349	(null)	(null)	(null)
12347	(null)	(null)	(null)
12351	(null)	(null)	(null)

Figura 20 – Resultado do RIGHT OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

Exemplo da sintaxe aplicando a cláusula USING, usado quando as colunas possuem o mesmo nome.

```
-- EXEMPLO RIGHT OUTER JOIN - PADRÃO SQL/99
-- CLÁUSULA USING
SELECT NR_MATRICULA,
       I.CD_PROJETO, I.DT_ENTRADA,I.DT_SAIDA
FROM  T_SIP_IMPLANTACAO I RIGHT OUTER JOIN T_SIP_FUNCIONARIO F
      USING ( NR_MATRICULA );
```

Código-fonte 18 – Exemplo de código-fonte SQL – RIGHT OUTER JOIN com USING
Fonte: Elaborado pela autora (2023)

Nota: Não aplique o qualificador na coluna “NR_MATRICULA”, pois acarretará um erro. Quando utilizamos RIGHT OUTER JOIN com a cláusula USING, não podemos utilizar qualificadores para a colunas chave primária/estrangeira.

1.5.3 Full Outer Join – Junções Externas

A junção do tipo **FULL OUTER JOIN** segue a **padronização ANSI** e implica, por sua vez, em criar a **junção externa entre duas tabelas,** é um tipo de junção externa. Esse tipo de junção surgiu a partir da padronização ANSI 99.

Nesse tipo de junção, todas as linhas da tabela à direita e à esquerda serão recuperadas, independentemente da existência de ocorrências relacionadas na tabela da esquerda ou da direita, ou seja, correspondência entre a chave primária e a chave estrangeira.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Preserva as linhas sem correspondência da primeira tabela (direita ou esquerda), juntando-as com uma linha nula na forma da segunda tabela (esquerda ou direita).

Denomina-se união regular (EQUIJOIN) aquelas uniões que têm a cláusula WHERE indicando a chave primária a estrangeira das tabelas afetadas pelo comando SELECT.

Podemos ilustrar a junção externa, através dos exemplos abaixo.

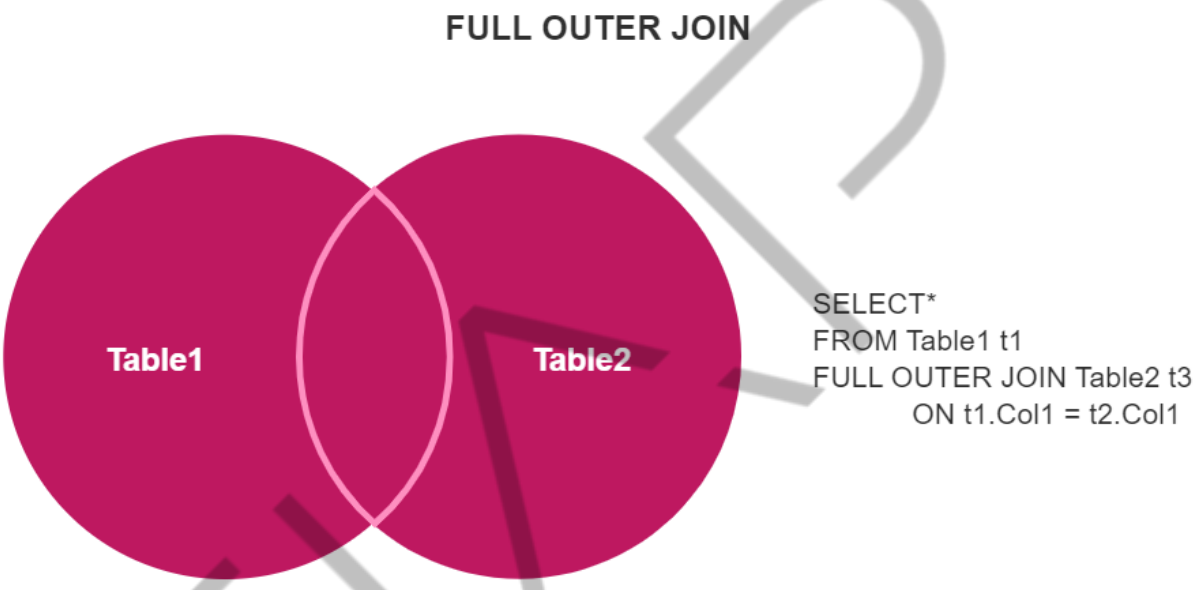


Figura 21 – FULL OUTER JOIN
Fonte: Google imagens (2017)

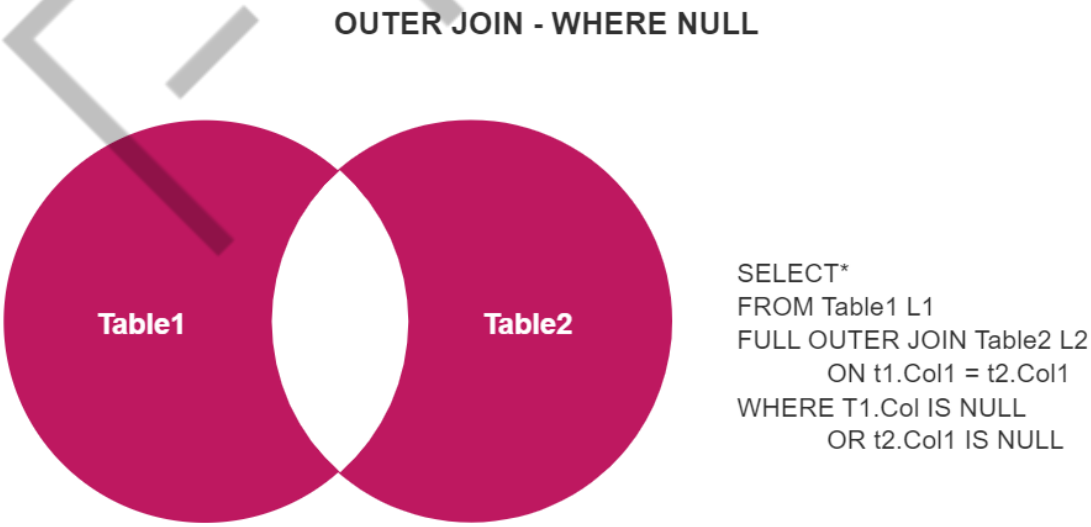


Figura 22 – FULL OUTER JOIN com WHERE NULL
Fonte: Google imagens (2017)

Utilizaremos o modelo abaixo, no qual uma empresa conveniada pode ter alunos associados (matriculados em uma academia) e alunos matriculados podem

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

pertencer a uma empresa conveniada. Como o relacionamento é opcional dos dois lados, conseguimos exemplificar a aplicação do FULL OUTER JOIN.

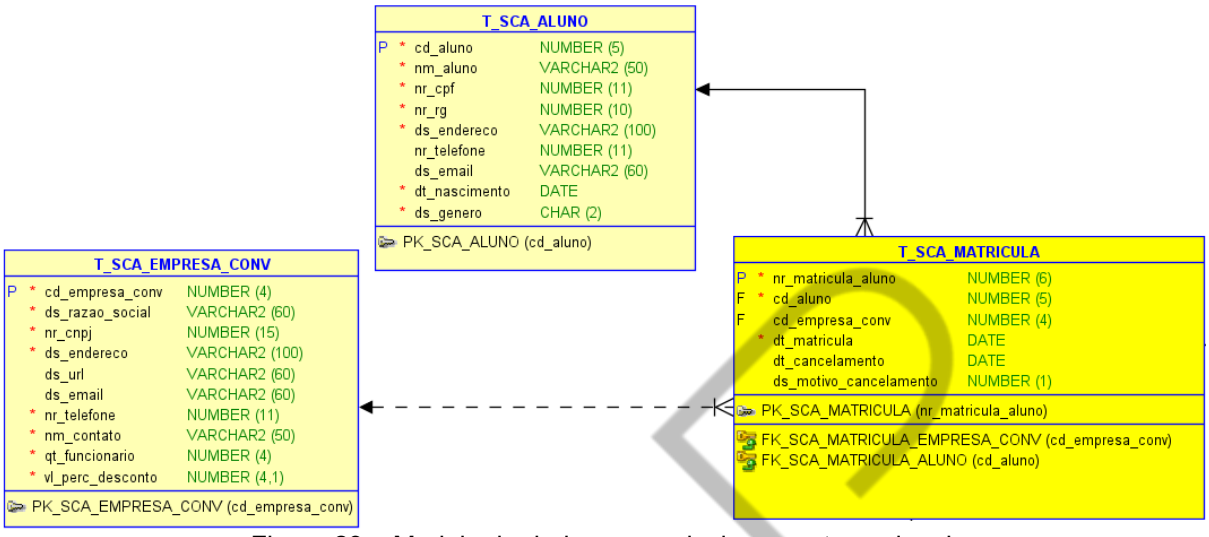


Figura 23 – Modelo de dados com relacionamento opcional
Fonte: Elaborado pela autora (2023)

Para exemplificar as aplicações do comando SELECT por meio da junção “FULL OUTER JOIN”, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_EXEMPLO_FULL_OUTER_JOIN.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados e populá-la em seguida.

```
-- CRIAÇÃO DO BANCO DE DADOS PARA EXEMPLIFICAR
-- FULL OUTER JOIN

-- CRIAÇÃO DA ESTRUTURA DO BANCO DE DADOS
/*
DROP TABLE T_SCA_EMPRESA_CONV CASCADE CONSTRAINT;
DROP TABLE T_SCA_ALUNO        CASCADE CONSTRAINT;
DROP TABLE t_sca_matricula    CASCADE CONSTRAINT;
*/

CREATE TABLE t_sca_aluno (
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
    cd_aluno        NUMBER(5) NOT NULL,
    nm_aluno        VARCHAR2(50) NOT NULL,
    nr_cpf          NUMBER(11) NOT NULL,
    nr_rg           NUMBER(10) NOT NULL,
    ds_endereco     VARCHAR2(100) NOT NULL,
    nr_telefone     NUMBER(11),
    ds_email        VARCHAR2(60),
    dt_nascimento   DATE NOT NULL,
    ds_genero       CHAR(2) NOT NULL
);

ALTER TABLE t_sca_aluno ADD CONSTRAINT pk_sca_aluno PRIMARY
KEY ( cd_aluno );

CREATE TABLE t_sca_empresa_conv (
    cd_empresa_conv   NUMBER(4) NOT NULL,
    ds_razao_social   VARCHAR2(60) NOT NULL,
    nr_cnpj           NUMBER(15) NOT NULL,
    ds_endereco       VARCHAR2(100) NOT NULL,
    ds_url            VARCHAR2(60),
    ds_email          VARCHAR2(60),
    nr_telefone       NUMBER(11) NOT NULL,
    nm_contato        VARCHAR2(50) NOT NULL,
    qt_funcionario    NUMBER(4) NOT NULL,
    vl_perc_desconto  NUMBER(4, 1) NOT NULL
);

ALTER     TABLE     t_sca_empresa_conv     ADD     CONSTRAINT
pk_sca_empresa_conv PRIMARY KEY ( cd_empresa_conv );

CREATE TABLE t_sca_matricula (
    nr_matricula_aluno      NUMBER(6) NOT NULL,
    cd_aluno                NUMBER(5) NOT NULL,
    cd_empresa_conv         NUMBER(4),
    dt_matricula            DATE NOT NULL,
    dt_cancelamento         DATE,
    ds_motivo_cancelamento  NUMBER(1)
);

ALTER TABLE t_sca_matricula ADD CONSTRAINT pk_sca_matricula
PRIMARY KEY ( nr_matricula_aluno );

ALTER TABLE t_sca_matricula
    ADD  CONSTRAINT  fk_sca_matricula_aluno  FOREIGN  KEY  (
cd_aluno )
        REFERENCES t_sca_aluno ( cd_aluno );

ALTER TABLE t_sca_matricula
    ADD CONSTRAINT fk_sca_matricula_empresa_conv FOREIGN KEY (
cd_empresa_conv )
        REFERENCES t_sca_empresa_conv ( cd_empresa_conv );
```

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
/*
DELETE FROM t_sca_empresa_conv;
DELETE FROM T_SCA_ALUNO;
DELETE FROM t_sca_matricula;
*/

-- INSERT´s PARA POPULAR AS TABELAS
INSERT INTO T_SCA_EMPRESA_CONV
       (   cd_empresa_conv,   ds_razao_social   ,   nr_cnpj,
ds_endereco,   ds_url,   ds_email,   nr_telefone,   nm_contato,
qt_funcionario, vl_perc_desconto )
VALUES ( 1, 'EMPRESA XYZ LTDA.' , 12345678901, 'Rua X, 25',
'http://xyz.com.br','XYZ@XYZ.COM.BR',     12345789,     'MARIA
APARECIDA',50, 3.5);

INSERT INTO T_SCA_EMPRESA_CONV
       (   cd_empresa_conv,   ds_razao_social   ,   nr_cnpj,
ds_endereco,   ds_url,   ds_email,   nr_telefone,   nm_contato,
qt_funcionario, vl_perc_desconto )
VALUES  (  2,  'EMPRESA  ABC  LTDA.'  ,  12345678912,'RUA  Z,
49','HTTP://ABC.COM.BR','ABC@ABC.COM.BR',   12121121,   'JOAO
XAVIER',520, 1.8);

INSERT INTO T_SCA_EMPRESA_CONV
       (   cd_empresa_conv,   ds_razao_social   ,   nr_cnpj,
ds_endereco,   ds_url,   ds_email,   nr_telefone,   nm_contato,
qt_funcionario, vl_perc_desconto )
VALUES  (  3,  'EMPRESA  BCD  LTDA.'  ,  12345678923,'RUA  F,
45','HTTP://BCD.COM.BR','BCD@BCD.COM.BR',  31313331,  'CARMEM
MIRANDA',85, 5.25);

INSERT INTO T_SCA_EMPRESA_CONV
       (   cd_empresa_conv,   ds_razao_social   ,   nr_cnpj,
ds_endereco,   ds_url,   ds_email,   nr_telefone,   nm_contato,
qt_funcionario, vl_perc_desconto )
VALUES ( 4, 'EMPRESA EFG LTDA.' , 12345678943,'RUA C, 59',
'HTTP://EFG.COM.BR','EFG@EFG.COM.BR',               43235686,
'DOMINGUINHOS',159, 3.9);

COMMIT;

SELECT * FROM T_SCA_EMPRESA_CONV;


INSERT INTO T_SCA_ALUNO
    (     CD_ALUNO,     NM_ALUNO,NR_CPF,     NR_RG,DS_ENDERECO,
NR_TELEFONE, DS_EMAIL, DT_NASCIMENTO, DS_GENERO )
VALUES
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
    ( 12345, 'MARIA DAS FLORES', 125612345,12347890,'RUA
X,77',        45629963        ,        'MARIA@MARIA.COM.BR',
TO_DATE('02/04/1985','DD/MM/YYYY'), 'M');

INSERT INTO T_SCA_ALUNO
    (    CD_ALUNO,    NM_ALUNO,NR_CPF,    NR_RG,DS_ENDERECO,
NR_TELEFONE, DS_EMAIL, DT_NASCIMENTO, DS_GENERO )
VALUES
    ( 12346,'JOÃO ROSA', 3223412345,32347890,'RUA, L 45',
945126321               ,                'JOAO@JOAO.COM.BR',
TO_DATE('12/04/1980','DD/MM/YYYY'),'H');

INSERT INTO T_SCA_ALUNO
    (    CD_ALUNO,    NM_ALUNO,NR_CPF,    NR_RG,DS_ENDERECO,
NR_TELEFONE, DS_EMAIL, DT_NASCIMENTO, DS_GENERO )
VALUES
    ( 12347, 'CAROLINA FERRAZ', 4223412345,42347890, 'RUA, K,
18',       945126354       ,       'CAROLINA@CAROLINA.COM.BR',
TO_DATE('27/09/1980','DD/MM/YYYY'),'F');

INSERT INTO T_SCA_ALUNO
     (    CD_ALUNO,    NM_ALUNO,NR_CPF,    NR_RG,DS_ENDERECO,
NR_TELEFONE, DS_EMAIL, DT_NASCIMENTO, DS_GENERO )
VALUES
    ( 12348, 'ROBERTO CARLOS', 5223452345,52345690,'RUA R,
89',        945123654       ,       'ROBERTO@ROBERTO.COM.BR',
TO_DATE('15/05/1990','DD/MM/YYYY'),'M');

INSERT INTO T_SCA_ALUNO
     (    CD_ALUNO,    NM_ALUNO,NR_CPF,    NR_RG,DS_ENDERECO,
NR_TELEFONE, DS_EMAIL, DT_NASCIMENTO, DS_GENERO )
VALUES
    ( 12349, 'SILVIO SANTOS', 6225612345,62345890,'RUA, A,
32',45623132        ,        'SILVIO@SILVIO.COM.BR'        ,
TO_DATE('19/11/1970','DD/MM/YYYY'),'M');

COMMIT;

SELECT * FROM T_SCA_ALUNO;


INSERT INTO T_SCA_MATRICULA
     (NR_MATRICULA_ALUNO    ,    CD_ALUNO,    CD_EMPRESA_CONV,
DT_MATRICULA, DT_CANCELAMENTO, DS_MOTIVO_CANCELAMENTO)
VALUES
     (1001,  12345,  1,  TO_DATE('02/10/2019','DD/MM/YYYY'),
NULL, NULL );

INSERT INTO T_SCA_MATRICULA
     (NR_MATRICULA_ALUNO    ,    CD_ALUNO,    CD_EMPRESA_CONV,
DT_MATRICULA, DT_CANCELAMENTO, DS_MOTIVO_CANCELAMENTO)
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
VALUES
     (1002,  12346,  1,  TO_DATE('05/05/2017','DD/MM/YYYY'),
NULL, NULL );

INSERT INTO T_SCA_MATRICULA
     (NR_MATRICULA_ALUNO    ,    CD_ALUNO,    CD_EMPRESA_CONV,
DT_MATRICULA, DT_CANCELAMENTO, DS_MOTIVO_CANCELAMENTO)
VALUES
     (1003,  12347,  2,  TO_DATE('01/02/2020','DD/MM/YYYY'),
NULL, NULL );

INSERT INTO T_SCA_MATRICULA
     (NR_MATRICULA_ALUNO    ,    CD_ALUNO,    CD_EMPRESA_CONV,
DT_MATRICULA, DT_CANCELAMENTO, DS_MOTIVO_CANCELAMENTO)
VALUES
     (1004, 12348, NULL, TO_DATE('15/09/2018','DD/MM/YYYY'),
NULL, NULL );

INSERT INTO T_SCA_MATRICULA
     (NR_MATRICULA_ALUNO    ,    CD_ALUNO,    CD_EMPRESA_CONV,
DT_MATRICULA, DT_CANCELAMENTO, DS_MOTIVO_CANCELAMENTO)
VALUES
     (1005, 12349, NULL, TO_DATE('17/03/2019','DD/MM/YYYY'),
NULL, NULL );

COMMIT;


SELECT * FROM T_SCA_MATRICULA;
```

Código-fonte 19 – Exemplo de código-fonte SQL – SCRIPT para criar estrutura exemplo FULL OUTER JOIN
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará à combinação (junção) das tabelas “EMPRESA_CONVENIADA” e “MATRICULA”, a partir de um FULL OUTER JOIN.

A condição de junção é estabelecida na cláusula ON, na qual comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção ou não entre as tabelas.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos ou não, ou seja, são recuperadas os registros (linhas) relacionados entre as duas tabelas e aqueles que não possuem associações. Em outras palavras, serão

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

recuperados todos os alunos que pertençam ou não a instituições, e instituições que têm ou não alunos.

Os alunos matriculados ou empresas conveniadas que não tiverem associações, no momento da recuperação, esses dados serão preenchidos com NULL.

Veremos o conteúdo de cada uma das tabelas separadamente e depois com a aplicação do FULL OUTER JOIN.

CD_EMPRESA_CONV	DS_RAZAO_SOCIAL	NR_CNPJ	DS_ENDERECO	DS_URL	DS_EMAIL	NR_TELEFONE	NM_CONTATO	QT_FUNCIONARIO	VL_PERC_DESCONTO
1	EMPRESA XYZ LTDA.	12345678901	Rua X, 25	http://x...	XYZ@X...	12345789	MARIA APARECIDA	50	3,5
2	EMPRESA ABC LTDA.	12345678912	RUA Z, 49	HTTP://A...	ABC@A...	12121121	JOAO XAVIER	520	1,8
3	EMPRESA BCD LTDA.	12345678923	RUA F, 45	HTTP://B...	BCD@B...	31313331	CARMEM MIRANDA	85	5,3
4	EMPRESA EFG LTDA.	12345678943	RUA C, 59	HTTP://E...	EFG@E...	43235686	DOMINGUINHOS	159	3,9

Figura 24 – Conteúdo armazenado na tabela EMPRESA CONVENIADA
Fonte: Elaborado pela autora (2023)

NR_MATRICULA_ALUNO	CD_ALUNO	CD_EMPRESA_CONV	DT_MATRICULA	DT_CANCELAMENTO	DS_MOTIVO_CANCELAMENTO
1001	12345	1	02/10/19	(null)	(null)
1002	12346	1	05/05/17	(null)	(null)
1003	12347	2	01/02/20	(null)	(null)
1004	12348	(null)	15/09/18	(null)	(null)
1005	12349	(null)	17/03/19	(null)	(null)

Figura 25 – Conteúdo armazenado na tabela MATRICULA
Fonte: Elaborado pela autora (2023)

```
-- EXEMPLO FULL OUTER JOIN - PADRÃO SQL/99
SELECT EC.CD_EMPRESA_CONV,
       EC.DS_RAZAO_SOCIAL,
       M.NR_MATRICULA_ALUNO,
       M.CD_ALUNO
  FROM T_SCA_EMPRESA_CONV EC FULL OUTER JOIN T_SCA_MATRICULA M
       ON (EC.CD_EMPRESA_CONV = M.CD_EMPRESA_CONV);
```

Código-fonte 20 – Exemplo de código-fonte SQL – FULL OUTER JOIN
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com FULL OUTER JOIN.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

CD_EMPRESA_CONV	DS_RAZAO_SOCIAL	NR_MATRICULA_ALUNO	CD_ALUNO
1	EMPRESA XYZ LTDA.	1001	12345
1	EMPRESA XYZ LTDA.	1002	12346
2	EMPRESA ABC LTDA.	1003	12347
(null)	(null)	1004	12348
(null)	(null)	1005	12349
4	EMPRESA EFG LTDA.	(null)	(null)
3	EMPRESA BCD LTDA.	(null)	(null)

Figura 26 – Resultado do FULL OUTER JOIN
Fonte: Elaborado pela autora (2023)

Podemos recuperar somente os alunos matriculados ou empresas conveniadas que não possuírem associações. Para isso, utilizaríamos o comando “SELECT” com FULL OUTER JOIN e a condição, na qual serão recuperadas apenas as combinações que resultaram em NULOS.

```
-- EXEMPLO COM FULL OUTER JOIN - PADRÃO SQL/99 -
-- COM VALIDAÇÃO DOS RESULTADOS NULOS
SELECT EC.CD_EMPRESA_CONV,
       EC.DS_RAZAO_SOCIAL,
       M.NR_MATRICULA_ALUNO,
       M.CD_ALUNO
FROM T_SCA_EMPRESA_CONV EC FULL OUTER JOIN T_SCA_MATRICULA M
       ON (EC.CD_EMPRESA_CONV = M.CD_EMPRESA_CONV)
 WHERE EC.CD_EMPRESA_CONV IS NULL OR
       M.NR_MATRICULA_ALUNO IS NULL;
```

Código-fonte 21 – Exemplo de código-fonte SQL – FULL OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

Resultado da execução do comando “SELECT” com FULL OUTER JOIN e recuperação apenas de NULOS.

CD_EMPRESA_CONV	DS_RAZAO_SOCIAL	NR_MATRICULA_ALUNO	CD_ALUNO
(null)	(null)	1004	12348
(null)	(null)	1005	12349
4	EMPRESA EFG LTDA.	(null)	(null)
3	EMPRESA BCD LTDA.	(null)	(null)

Figura 27 – Resultado do FULL OUTER JOIN, recuperando NULOS
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Exemplo da sintaxe aplicando a cláusula USING, usado quando as colunas possuem o mesmo nome.

```
-- EXEMPLO FULL OUTER JOIN - PADRÃO SQL/99
-- CLÁUSULA USING
SELECT CD_EMPRESA_CONV,
       EC.DS_RAZAO_SOCIAL,
       M.NR_MATRICULA_ALUNO,
       M.CD_ALUNO
FROM T_SCA_EMPRESA_CONV EC FULL OUTER JOIN T_SCA_MATRICULA M
       USING (CD_EMPRESA_CONV);
```

Código-fonte 22 – Exemplo de código-fonte SQL – FULL OUTER JOIN com USING
Fonte: Elaborado pela autora (2017)

Nota: Não aplique o qualificador na coluna "CD_INSTITUICAO", pois acarretará um erro. Quando utilizamos FULL OUTER JOIN com a cláusula USING, não podemos utilizar qualificadores para a colunas chave primária/estrangeira.

1.6 Self Join ou Autojunção

As junções do tipo **SELF JOIN** são utilizadas em casos que temos no modelo de dados, a figura do **auto-relacionamento (relacionamento recursivo).** Nesse caso, precisamos acessar a mesma tabela duas vezes – uma para recuperar os registros e a outra para buscar os dados relacionados a ela (auto-relacionamento).

Utilizaremos o modelo abaixo, em que cada funcionário pode ter a referência de um gerente, ou seja, pode ser gerenciado por um outro funcionário que ocupa um cargo de gerência. Observe que a tabela "FUNCIONARIO" se associa com ela mesma. Com este exemplo representaremos o SELF JOIN.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

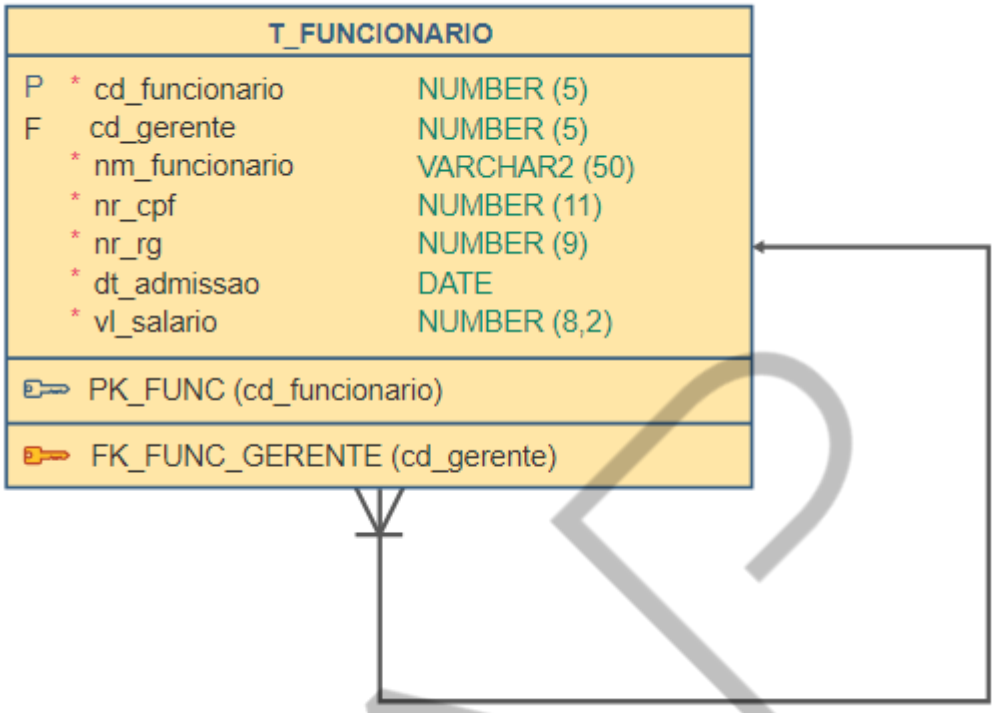


Figura 28 – Modelo de dados, autorrelacionamento
Fonte: Elaborado pela autora (2017)

Para exemplificar as aplicações utilizando SELF JOIN, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_EXEMPLO_SELF_JOIN.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados para esta aplicação.

```
-- EXEMPLO DE ESTRUTURA PARA APLICAR SELF JOIN

-- DROP TABLE T_EXE_FUNCIONARIO CASCADE CONSTRAINT;


CREATE TABLE T_EXE_FUNCIONARIO

(
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
   cd_funcionario  NUMBER (5)     NOT NULL ,
   cd_gerente      NUMBER (5)         NULL ,
   nm_funcionario  VARCHAR2 (50) NOT NULL ,
   nr_cpf          NUMBER (11)    NOT NULL ,
   nr_rg           NUMBER (9)     NOT NULL ,
   dt_admissao     DATE           NOT NULL ,
   vl_salario      NUMBER (8,2)   NOT NULL
) ;
/* Chave primária da tabela Funcionario */
ALTER TABLE T_EXE_FUNCIONARIO
ADD CONSTRAINT PK_EXE_FUNCIONARIO
PRIMARY KEY ( cd_funcionario ) ;

/******* Chave ESTRANGEIRA - Tabela Funcionario ********/
ALTER TABLE T_EXE_FUNCIONARIO
ADD CONSTRAINT FK_EXE_GERENTE
   FOREIGN KEY (cd_gerente)
REFERENCES T_EXE_FUNCIONARIO (cd_funcionario);

-------------------------------------------------
-- EXEMPLO DE INSERÇÕES
--------------------------------------------------
INSERT INTO T_EXE_FUNCIONARIO
VALUES (1,NULL,'RITA',12345678901,11222333,
TO_DATE('15/07/2021','DD/MM/YYYY'), 4567.98);
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
INSERT INTO T_EXE_FUNCIONARIO
VALUES (2,NULL,'JOAO',12342233445,11333444,
TO_DATE('28/03/2019','DD/MM/YYYY'), 3894.57);

INSERT INTO T_EXE_FUNCIONARIO
VALUES (3,NULL,'ROSA',12234543441,11444555,
TO_DATE('03/04/2017','DD/MM/YYYY'), 1452.25);

INSERT INTO T_EXE_FUNCIONARIO
VALUES (4,NULL,'MARIA',1555566651,116667778,
TO_DATE('29/07/2020','DD/MM/YYYY'), 1598.47);

INSERT INTO T_EXE_FUNCIONARIO
VALUES (5,NULL,'ANA'  ,1343322901,119764333,
TO_DATE('12/04/2021','DD/MM/YYYY'), 3457.33);

-------------------------------------------------
-- INFORMAR OS GERENTES DE CADA FUNCIONARIO
-- ATRAVES DO UPDATE A INDICAÇÃO DOS GERENTES
-- DE CADA FUNCIONARIO
UPDATE T_EXE_FUNCIONARIO
   SET cd_gerente=3
 WHERE cd_funcionario IN (1,2,5);

COMMIT;
SELECT * FROM T_EXE_FUNCIONARIO ;
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Código-fonte 23 – Exemplo de código-fonte SQL – Script para criar a estrutura para o SELF JOIN
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará à combinação (junção) da tabela “FUNCIONARIO” com ela mesma, SELF JOIN.

A condição de junção é estabelecida na cláusula ON, em que comparamos a chave primária a chave estrangeira com o objetivo de recuperar a intersecção entre a própria tabela.

Os registros (linhas) recuperados são aqueles que atendem a intersecção dos conjuntos, ou seja, são recuperados os registros (linhas) relacionados entre a própria tabela. Em outras palavras, serão recuperados todos os funcionários com seus respectivos gerentes.

CD_FUNCIONARIO	CD_GERENTE	NM_FUNCIONARIO	NR_CPF	NR_RG	DT_ADMISSAO	VL_SALARIO
1	3	RITA	12345678901	11222333	15/07/21	4567,98
2	3	JOAO	12342233445	11333444	28/03/19	3894,57
3	(null)	ROSA	12234543441	11444555	03/04/17	1452,25
4	(null)	MARIA	1555566651	116667778	29/07/20	1598,47
5	3	ANA	1343322901	119764333	12/04/21	3457,33

Figura 29 – Conteúdo armazenado na tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

```
-- EXEMPLO AUTOJUNÇAO (SELF JOIN) - PADRÃO SQL/99
SELECT FUNC.CD_FUNCIONARIO, FUNC.NM_FUNCIONARIO,
       FUNC.CD_GERENTE, GER.NM_FUNCIONARIO "GERENTE DO FUNC"
  FROM T_EXE_FUNCIONARIO FUNC INNER JOIN T_EXE_FUNCIONARIO GER
       ON (GER.CD_FUNCIONARIO = FUNC.CD_GERENTE);
```

Código-fonte 24 – Exemplo de código-fonte SQL – SELF JOIN
Fonte: Elaborado pela autora (2023)

Nota: É necessário o uso do apelido nas tabelas e nomes das colunas sempre que tratarmos de SELF JOIN, em função de estarmos apontando para a mesma tabela. Ocorrerá erro, caso os apelidos não sejam utilizados.

Resultado da execução do comando “SELECT” com SELF JOIN.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

CD_FUNCIONARIO	NM_FUNCIONARIO	CD_GERENTE	GERENTE DO FUNC
1	RITA	3	ROSA
2	JOAO	3	ROSA
5	ANA	3	ROSA

Figura 30 – Resultado do SELF JOIN
Fonte: Elaborado pela autora (2023)

1.7 Non-equijoins – Junções não idênticas

União de tabelas sem colunas em comum (non-equijoin) é um tipo de junção que não requer a comparação entre as chaves primária e estrangeira.

Existem situações em que mesmo não havendo um relacionamento entre as tabelas, há o relacionamento de uma coluna com o intervalo de outras colunas em outras tabelas.

Exemplos:

- Os empregados estão associados a uma faixa salarial. Caso criemos as tabelas relacionadas, quando houver mudança de faixa, teremos de ir na tabela de empregados para alterar a faixa salarial (FK) do empregado e vice-versa; quando alterar o salário do empregado, precisamos buscar a nova faixa e associá-la novamente ao empregado.
- Faixa de imposto de renda, alíquotas de impostos em geral, enquadramentos baseados em valores e ou quantidades são exemplos típicos de consultas non-equijoins, e nesse caso, as tabelas não seriam relacionadas através do modelo e sim da aplicação.

Utilizaremos o modelo abaixo, em que o preço de um CD pertence a uma faixa de preço. Observe que não há um relacionamento, todas as vezes que o CD mudar de preço, pode mudar a faixa, portanto a relação (chave estrangeira) mudaria também.

Para evitar a manutenção do relacionamento, podemos consultar as faixas de preços e verificar em que ponto o preço do CD se enquadra, desta forma, estamos fazendo uma junção não idêntica, uma vez que não estamos comparando chave primária e chave estrangeira, e sim as colunas “VL_PRECO_VENDA” com as colunas “VL_INICIAL” e “VL_FINAL”.

Com este exemplo, representaremos uma junção não idêntica.

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

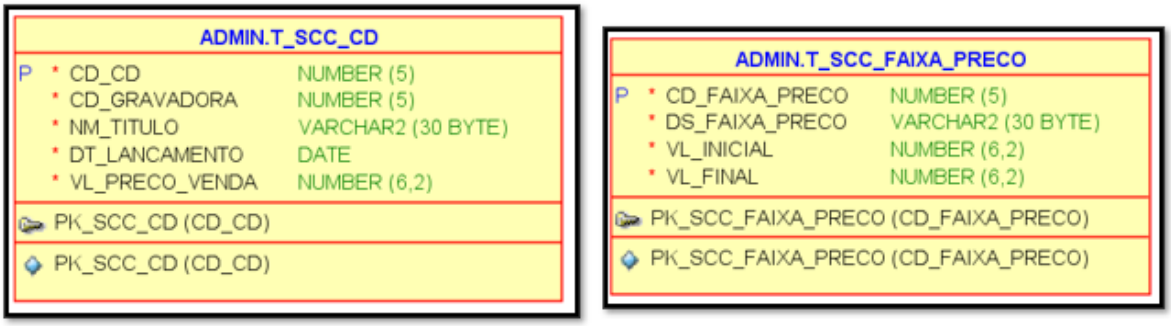


Figura 31 – Modelo de dados, sem relacionamento entre as tabelas
Fonte: Elaborado pela autora (2023)

Para exemplificar as aplicações utilizando non-equijoin, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_EXEMPLO_NON-EQUIJOIN.SQL”, com a extensão SQL.

Apresentamos abaixo o código na linguagem SQL, que permite implementar a estrutura de banco de dados para esta aplicação.

```
-- EXEMPLO DE ESTRUTURA PARA APLICAR NON-EQUIJOIN
/*
DROP TABLE T_SCC_CD CASCADE  CONSTRAINT;
DROP TABLE T_SCC_FAIXA_PRECO CASCADE CONSTRAINT;
*/

CREATE TABLE T_SCC_CD
(
   cd_cd           NUMBER (5)      NOT NULL ,
   cd_gravadora    NUMBER (5)      NOT NULL ,
   nm_titulo       VARCHAR2 (30)   NOT NULL ,
   dt_lancamento   DATE            NOT NULL,
   vl_preco_venda  NUMBER (6,2)    NOT NULL
);

ALTER TABLE T_SCC_CD
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
ADD CONSTRAINT PK_SCC_CD
   PRIMARY KEY ( cd_cd ) ;

CREATE TABLE T_SCC_FAIXA_PRECO
(
   cd_faixa_preco NUMBER (5)    NOT NULL ,
   ds_faixa_preco VARCHAR2 (30) NOT NULL ,
   vl_inicial     NUMBER (6,2)  NOT NULL ,
   vl_final       NUMBER (6,2)  NOT NULL
);

ALTER TABLE T_SCC_FAIXA_PRECO
ADD CONSTRAINT PK_SCC_FAIXA_PRECO
PRIMARY KEY ( cd_faixa_preco ) ;


/*
DELETE FROM T_SCC_CD;
DELETE FROM T_SCC_FAIXA_PRECO;
*/

-- INSERT´s PARA POPULAR A TABELA
INSERT INTO T_SCC_CD
(    cd_cd,    cd_gravadora    ,    nm_titulo,    dt_lancamento,
vl_preco_venda )
VALUES ( 1, 101 , 'CD ABC', TO_DATE('10/07/2015','DD/MM/YYYY')
, 15.50);

INSERT INTO T_SCC_CD
(    cd_cd,    cd_gravadora    ,    nm_titulo,    dt_lancamento,
vl_preco_venda )
VALUES ( 2, 101 , 'CD DEF', TO_DATE('21/03/2016','DD/MM/YYYY')
, 23.40);
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
INSERT INTO T_SCC_CD
(    cd_cd,    cd_gravadora    ,    nm_titulo,    dt_lancamento,
vl_preco_venda )
VALUES ( 3, 101 , 'CD GHI', TO_DATE('19/10/2016','DD/MM/YYYY')
, 56.75);

INSERT INTO T_SCC_CD
(    cd_cd,    cd_gravadora    ,    nm_titulo,    dt_lancamento,
vl_preco_venda )
VALUES ( 4, 101 , 'CD KLM', TO_DATE('13/03/2015','DD/MM/YYYY')
, 33.90);

COMMIT;
SELECT * FROM T_SCC_CD;

INSERT INTO T_SCC_FAIXA_PRECO
( cd_faixa_preco, ds_faixa_preco, vl_inicial, vl_final )
VALUES ( 1, 'SELO VERDE', 5.00, 15.99);

INSERT INTO T_SCC_FAIXA_PRECO
( cd_faixa_preco, ds_faixa_preco, vl_inicial, vl_final )
VALUES ( 2, 'SELO AZUL', 16.00, 25.99);

INSERT INTO T_SCC_FAIXA_PRECO
( cd_faixa_preco, ds_faixa_preco, vl_inicial, vl_final
VALUES ( 3, 'SELO AMARELO', 26.00, 45.99);

INSERT INTO T_SCC_FAIXA_PRECO
( cd_faixa_preco, ds_faixa_preco, vl_inicial, vl_final )
VALUES ( 4, 'SELO VERMELHO', 46.00, 65.99);

INSERT INTO T_SCC_FAIXA_PRECO
( cd_faixa_preco, ds_faixa_preco, vl_inicial, vl_final )
VALUES ( 5, 'SELO PRETO', 66.00, 100.00);
```

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
COMMIT;
SELECT * FROM T_SCC_FAIXA_PRECO;
```

Código-fonte 25 – Exemplo de código-fonte SQL – Script para criar estrutura para NON-EQUIJOIN
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará à combinação (junção) das tabelas “CD” e “FAIXA_PRECO” a partir de uma junção não idêntica (NON-EQUIJOIN).

A condição de junção é estabelecida na cláusula WHERE, na qual comparamos os campos que possuem uma faixa de valores correspondentes.

Os registros (linhas) recuperados são aqueles que atendem a condição estabelecida na cláusula WHERE. Em outras palavras, serão recuperados todos os CDs com suas respectivas faixas de preço.

CD_CD	CD_GRAVADORA	NM_TITULO	DT_LANCAMENTO	VL_PRECO_VENDA
1	101	CD ABC	10/07/15	15,5
2	101	CD DEF	21/03/16	23,4
3	101	CD GHI	19/10/16	56,75
4	101	CD KLM	13/03/15	33,9

Figura 32 – Conteúdo armazenado na tabela CD
Fonte: Elaborado pela autora (2023)

CD_FAIXA_PRECO	DS_FAIXA_PRECO	VL_INICIAL	VL_FINAL
1	SELO VERDE	5	15,99
2	SELO AZUL	16	25,99
3	SELO AMARELO	26	45,99
4	SELO VERMELHO	46	65,99
5	SELO PRETO	66	100

Figura 33 – Conteúdo armazenado na tabela FAIXA_PRECO
Fonte: Elaborado pela autora (2023)

```
-- EXEMPLO NON-EQUIJOIN
SELECT CD.NM_TITULO, CD.VL_PRECO_VENDA, FP.DS_FAIXA_PRECO
  FROM T_SCC_CD CD, T_SCC_FAIXA_PRECO FP
 WHERE CD.VL_PRECO_VENDA
        BETWEEN FP.VL_INICIAL AND FP.VL_FINAL;
```

Código-fonte 26 – Exemplo de código-fonte SQL – NON-EQUIJOIN
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Resultado da execução do comando “SELECT” com NON-EQUIJOIN.

NM_TITULO	VL_PRECO_VENDA	DS_FAIXA_PRECO
CD GHI	56,75	SELO VERMELHO
CD KLM	33,9	SELO AMARELO
CD DEF	23,4	SELO AZUL
CD ABC	15,5	SELO VERDE

Figura 34 – Resultado do NON-EQUIJOIN
Fonte: Elaborado pela autora (2023)

1.8 Hands On – Consulta ou Recuperação de dados utilizando JOINS

1.8.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS

1.8.1.1 Recuperando dados utilizando JOINS

No item 1.3, utilizamos o “SCRIPT DDL” e “SCRIPT DML”, para criar a estrutura do estudo de caso SIP e popular as tabelas deste projeto.

Iremos consultar os dados das tabelas deste estudo de caso, para fixarmos os comandos trabalhados nas seções anteriores.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DRS-JOIN_PROJETO_SIP.SQL”, com a extensão SQL.

Consulta 1

Escreva e execute a instrução SQL para realizar a consulta nas tabelas DEPARTAMENTO, FUNCIONÁRIO E DEPENDENTE.

Liste todos os funcionários que possuam dependentes.

A lista deve ser apresentada de forma ordenada por nome do funcionário, em ordem crescente e nome do dependente, em ordem crescente.

Mostrar: número da matrícula, nome do funcionário, nome do departamento, nome do dependente e data de nascimento do dependente.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Utilize apelidos e qualificadores para as tabelas.

Utilize o conceito de junção de tabelas.

```
-- CONSULTA 1
  SELECT F.NR_MATRICULA, F.NM_FUNCIONARIO,
         DP.NM_DEPTO, D.NM_DEPENDENTE, D.DT_NASCIMENTO
   FROM  T_SIP_DEPARTAMENTO DP INNER JOIN  T_SIP_FUNCIONARIO F
         ON (DP.CD_DEPTO = F.CD_DEPTO)
         INNER JOIN T_SIP_DEPENDENTE D
         ON (F.NR_MATRICULA = D.NR_MATRICULA)
ORDER BY F.NM_FUNCIONARIO ASC, D.NM_DEPENDENTE ASC;
```

Código-fonte 27 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 1
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

NR_MATRICULA	NM_FUNCIONARIO	NM_DEPTO	NM_DEPENDENTE	DT_NASCIMENTO
12345	JOAO DA SILVA	FINANCEIRO	JOANINHA	12/10/10
12345	JOAO DA SILVA	FINANCEIRO	JULINHA	15/10/12
12345	JOAO DA SILVA	FINANCEIRO	TONINHO	22/10/14
12359	JOSE DE CAMARGO	RECURSOS HUMANOS	CARMINHA	18/06/18
12359	JOSE DE CAMARGO	RECURSOS HUMANOS	LUIZINHA	21/06/16
12359	JOSE DE CAMARGO	RECURSOS HUMANOS	MARIAZINHA	11/06/14
12356	THIAGO DE ALMEIDA	FATURAMENTO	JUNINHO	02/10/11
12356	THIAGO DE ALMEIDA	FATURAMENTO	MARCELINHO	17/10/15
12356	THIAGO DE ALMEIDA	FATURAMENTO	ZEZINHO	15/10/13

Figura 35 – Resultado da Consulta 1
Fonte: Elaborado pela autora (2023)

Consulta 2

Escreva e execute a instrução SQL para realizar a consulta nas tabelas PROJETO, IMPLANTACAO E FUNCIONARIO.

Liste todos os projetos que possuam implantações ainda não finalizadas.

A lista deve ser apresentada de forma ordenada por nome do projeto e nome do funcionário, em ordem crescente.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Mostrar: código do projeto, nome do projeto, data de início, código da implantação, nome do funcionário, data de entrada e data de saída do funcionário do projeto.

Utilize apelidos e qualificadores para as tabelas.

Utilize o conceito de junção de tabelas.

```
-- CONSULTA 2
  SELECT   P.CD_PROJETO, P.NM_PROJETO,
           P.DT_INICIO, I.CD_IMPLANTACAO,
           F.NM_FUNCIONARIO, I.DT_ENTRADA, I.DT_SAIDA
    FROM   T_SIP_PROJETO P INNER JOIN  T_SIP_IMPLANTACAO I
           ON ( P.CD_PROJETO = I.CD_PROJETO )
           INNER JOIN   T_SIP_FUNCIONARIO F
           ON ( F.NR_MATRICULA = I.NR_MATRICULA )
   WHERE   I.DT_SAIDA IS  NULL
ORDER BY   P.NM_PROJETO ASC, F.NM_FUNCIONARIO ASC;
```

Código-fonte 28 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 2
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

CD_PROJETO	NM_PROJETO	DT_INICIO	CD_IMPLANTACAO	NM_FUNCIONARIO	DT_ENTRADA	DT_SAIDA
10001	PROJETO 1	11/10/16	2	MANUEL DA SILVA	11/10/16	(null)
10003	PROJETO 3	28/04/12	3	JOAO DA SILVA	28/04/13	(null)
10003	PROJETO 3	28/04/12	1	JOSEFINA DE ALMEIDA	28/04/13	(null)
10003	PROJETO 3	28/04/12	2	LUCIANA DE ALMEIDA	28/04/13	(null)
10004	PROJETO 4	25/03/17	2	ANTONIO DE CAMARGO	25/03/17	(null)
10004	PROJETO 4	25/03/17	1	LARISSSA DE CAMARGO	25/03/17	(null)

Figura 36 – Resultado da Consulta 2
Fonte: Elaborado pela autora (2023)

Consulta 3

Escreva e execute a instrução SQL para realizar a consulta nas tabelas FUNCIONARIO E DEPENDENTE.

Liste todos os funcionários que não possuam dependentes (apenas os que não possuam dependentes).

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

A lista deve ser apresentada de forma ordenada por nome do funcionário, em ordem crescente.

Mostrar: número de matrícula, nome do funcionário e nome do dependente.

Utilize apelidos e qualificadores para as tabelas.

Utilize o conceito de junção de tabelas.

Utilize Left Outer Join.

```
-- CONSULTA 3
  SELECT  F.NR_MATRICULA,
          F. NM_FUNCIONARIO,
          D.NM_DEPENDENTE,
          D.DT_NASCIMENTO
FROM T_SIP_FUNCIONARIO F LEFT OUTER JOIN  T_SIP_DEPENDENTE  D
          ON ( F.NR_MATRICULA = D.NR_MATRICULA )
   WHERE  D.NR_MATRICULA IS NULL
ORDER BY  F. NM_FUNCIONARIO;
```

Código-fonte 29 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 3
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

NR_MATRICULA	NM_FUNCIONARIO	NM_DEPENDENTE	DT_NASCIMENTO
12350	ALFREDO DE SOUZA	(null)	(null)
12358	ANTONIO DE CAMARGO	(null)	(null)
12351	GISELE DE JESUS	(null)	(null)
12347	JANDIRA DA SILVA	(null)	(null)
12354	JOSEFINA DE ALMEIDA	(null)	(null)
12348	KATIA REGINA SOUZA	(null)	(null)
12357	LARISSSA DE CAMARGO	(null)	(null)
12355	LUCIANA DE ALMEIDA	(null)	(null)
12346	MANUEL DA SILVA	(null)	(null)
12349	MARIA DAS DORES SOUZA	(null)	(null)
12352	RAFAEL DE JESUS	(null)	(null)
12353	ROSANA DE JESUS	(null)	(null)

Figura 37 – Resultado da Consulta 3
Fonte: Elaborado pela autora (2023)

Consulta 4

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Escreva e execute a instrução SQL, para realizar a consulta nas tabelas FUNCIONARIO E IMPLANTACAO.

Liste todos os funcionários que não participam de implantações.

A lista deve ser apresentada de forma ordenada por nome do funcionário e código da implantação, em ordem crescente.

Mostrar: número de matrícula, nome do funcionário, código da implantação e data de entrada.

Utilize apelidos e qualificadores para as tabelas.

Utilize o conceito de junção de tabelas.

Utilize Left Outer Join.

```
-- CONSULTA 4
  SELECT F.NR_MATRICULA,
         F.NM_FUNCIONARIO,
         I.CD_IMPLANTACAO ,
         I.DT_ENTRADA
  FROM T_SIP_FUNCIONARIO F LEFT OUTER JOIN T_SIP_IMPLANTACAO I
         ON ( F.NR_MATRICULA = I.NR_MATRICULA )
   WHERE I.NR_MATRICULA IS NULL
ORDER BY F. NM_FUNCIONARIO, I.CD_IMPLANTACAO ;
```

Código-fonte 30 – Exemplo de código-fonte SQL – comando SELECT com JOIN – Consulta 4
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

NR_MATRICULA	NM_FUNCIONARIO	CD_IMPLANTACAO	DT_ENTRADA
12351	GISELE DE JESUS	(null)	(null)
12347	JANDIRA DA SILVA	(null)	(null)
12359	JOSE DE CAMARGO	(null)	(null)
12349	MARIA DAS DORES SOUZA	(null)	(null)
12353	ROSANA DE JESUS	(null)	(null)
12356	THIAGO DE ALMEIDA	(null)	(null)

Figura 38 – Resultado da Consulta 4
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C.A. **Projeto de Banco de Dados**. Série Livros Didáticos. v. 4. 6. ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

PRICE, J. **Oracle Database 11g SQL** São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados**: Implementação em SQL, PL/SQL e Oracle 11g. São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

GLOSSÁRIO

DRS	Data Retrieve Language (Linguagem de recuperação de dados)
DQL	Data Query Language (Linguagem para consulta de dados)
SELECT	Comando da linguagem SQL, responsável por permitir a consulta ou recuperação de registros ou linhas nas tabelas.
EQUIJOINS	Utilizam o operador de igualdade (=). São consideradas como consequência de um modelo relacional, ou seja, espelham a relação dos dados representados em um modelo E-R. São as relações em que temos a presença das chaves primária e estrangeira.
NON-EQUIJOINS	Utilizam um operador que não é o de igualdade, como <, >, BETWEEN etc. São consideradas como junções atípicas, em que não há a presença de chave primária e nem mesmo chaves estrangeiras. Os relacionamentos são feitos por meio de colunas, normalmente não chaves primárias e nem únicas, mas que possuem algum tipo de relação associada à necessidade de negócio, para que possamos expressar a junção.
JOINS INTERNAS	Retornam um registro/linha somente quando as colunas da junção possuírem valores que satisfaçam a condição da junção.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

JOINS EXTERNAS	Retornam um registro/linha mesmo quando uma das colunas na condição de junção possuir um valor nulo.
AUTOJOINS	Retornam registros/linhas, possuem valores associados na mesma tabela.

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com